

Kiel University of Applied Sciences

Faculty of Media

Master Thesis

Data-Driven Evaluation and Optimization of a Multi-Agent RAG Chatbot with Automated Trust Scoring

Submitted by:	Vanapala Sai Mohit
Registration number:	946644
Degree Program:	Data Science
Submission Date:	November 03, 2026
Primary Examiner:	Prof. Dr. Stephan Dorfel
Secondary Examiner:	Guedria Mohamed Amine
External organization:	Bosch Rexroth AG

Declaration

NBl. HS MSGWG Schl.-H. Nr. 6/2016 vom 20. Dezember 2016, S. 102

Tag der Bekanntmachung auf der Internetseite der Fachhochschule Kiel: 20. Oktober 2016

37 Anlage D (zu § 28 PVO)

Name: Vanapala Sai Mohit

Martikel-Nr.: 946644

Studiengang: Data Science

Ich versichere, dass ich die Masterarbeit “Data-Driven Evaluation and Optimization of a Multi-Agent RAG Chatbot with Automated Trust Scoring” selbständig und ohne unzulässige fremde Hilfe angefertigt habe und dass ich alle von anderen Autoren wörtlich übernommenen Stellen wie auch die sich an die Gedankengänge anderer Autoren eng anlehnenen Ausführungen meiner Arbeit besonders gekennzeichnet und die entsprechenden Quellen angegeben habe. Diese Arbeit hat noch keiner Prüfungsbehörde vorgelegen.

Ort, Datum:

Unterschrift:

I certify that I have completed the Master’s thesis “Data-Driven Evaluation and Optimization of a Multi-Agent RAG Chatbot with Automated Trust Scoring” independently and without unauthorized outside help and that I have specifically marked all passages taken verbatim from other authors as well as those closely based on the ideas of other authors in my work and have cited the corresponding sources. This thesis has not yet been submitted to any examination authority.

Place, Date:

Signature:

Acknowledgments

I would like to thank my supervisor, Guedria Mohamed Amine, for his guidance and support throughout this thesis. He gave me the context and background I needed to get started, connected me with the right people, and provided the resources to carry the project through to completion. His trust in the work and his feedback at every stage made the process a smooth one.

I am grateful to Prof. Dr. Stephan Doerfel for his academic supervision, for the opportunity to present my progress regularly, and for the direction he offered at each of those discussions.

My thanks also go to the team at Bosch Rexroth. Their technical background on how the company operates shaped the quality of this work, and the brainstorming sessions, discussions and reviews along the way were central to how the system took form.

Finally, I would like to thank the University of Applied Sciences Kiel for the opportunity to carry out this thesis, which gave me a close look at how production RAG systems are built and where AI genuinely fits into an industrial setting.

Contents

1	Introduction	1
2	Scope	4
3	Literature Review	5
3.1	Foundations of Retrieval-Augmented Generation	5
3.2	Retrieval with Embeddings, Graphs and Reranking	5
3.3	Trust and Reliability of Source Documents	7
3.4	Evaluation and Observability of RAG Systems	8
3.5	Research Gap	9
4	Theory	11
4.1	LLM and RAG Background	11
4.2	Retrieval Methods	12
4.2.1	Vector Retrieval	12
4.2.2	Graph Retrieval	12
4.2.3	Hybrid Retrieval	14
4.3	Trust Score	15
4.4	Evaluation Metrics	16
4.4.1	Retrieval Evaluation Metrics	16
4.4.2	Generator Evaluation Metrics	17
4.5	Observability	19
4.6	AVAILABLE 1.0	20
5	Methodology	22
5.1	System Overview and Architecture	22
5.2	Data Preparation and Cleaning	25
5.2.1	Data Source	26
5.2.2	Data Processing	26
5.2.3	Bronze Layer	27
5.2.4	Silver Layer	28
5.2.5	Gold Layer	31
5.3	Retrieval Implementation	32
5.3.1	Embedding	32
5.3.2	Vector Database	34
5.3.3	Vector Eye	34

5.3.4	Graph Eye	35
5.3.5	Hybrid Retrieval	42
5.4	Trust Scoring Implementation	44
5.4.1	The Prior	45
5.4.2	The Support Score	46
5.4.3	Limits of the Method and the Choice of Weights	47
5.4.4	Use at Query Time	48
5.5	Evaluation	48
5.5.1	Evaluation Datasets	48
5.5.2	Retriever Evaluation	50
5.5.3	Generator Evaluation	51
5.5.4	Trust Score Evaluation	51
5.6	Observability	54
5.6.1	Guidelines for Trustworthy Documents	55
6	Experiments and Results	57
6.1	Experimental Datasets	57
6.2	Retrieval Results	57
6.2.1	Graph Engine Selection	58
6.2.2	Vector, Graph and Hybrid Retrieval	59
6.3	Generation Results	60
6.3.1	AVAILABLE 1.0 versus AVAILABLE 2.0	61
6.4	Generation Results	63
6.5	Trust Score Validation	66
6.5.1	Experiment 1, Detecting a Loss of Quality	66
6.5.2	Experiment 2, Helping the RAG	68
6.5.3	Why the Full Score Trails the Prior	71
6.6	Observability Dashboard	72
7	Discussion	73
8	Conclusion	74
9	Future Work	75
	Bibliography	76
	List of Figures	80
	List of Tables	82

Abstract

Manufacturing enterprises accumulate large training and technical knowledge bases over years of operation, in which critical information is hard to locate and harder to trust. Retrieval-augmented generation (RAG) makes such content conversationally accessible, but conventional RAG optimizes relevance alone and offers no signal for source reliability. This limitation confines deployments to manually pre-vetted corpora and prevents transfer to organically grown ones. This thesis presents AVAILABLE 2.0, an industrial knowledge assistant that unifies relevance and reliability within a single graph-based mechanism. Hybrid retrieval fuses dense vector search with Personalized PageRank over an entity–chunk graph, while an automated trust score propagates metadata-derived priors of source, authorship, freshness and uniqueness across the same graph, so that both signals arise from one propagation operator and are fused at ranking time. The system was evaluated against its deployed predecessor on an industrial training corpus spanning more than 300 courses and a benchmark of 1215 questions. Hybrid retrieval raised correct-source identification from 83.2% to 93.6% and context precision from 0.37 to 0.94, with gains concentrated on procedural questions, where single-vector retrieval fails most. In a knowledge-poisoning experiment, trust-aware ranking increased the selection of trustworthy sources from 38% to 71% and reduced answers grounded in corrupted sources from 31% to 4%. The results show that automated source-level trust scoring is a precondition for extending LLM-based knowledge assistance from curated pilot corpora to the grown knowledge bases of production environments.

1 Introduction

Technology changes faster today than at any point before. An important invention once took many years to spread and become part of everyday work, but that cycle has grown much shorter, and artificial intelligence has shortened it further. A skill learned today can already be outdated by the time it is applied. The World Economic Forum estimates that around 39% of the skills workers hold today will change or lose their value between 2025 and 2030 [1].

This shift is visible across industry. Many companies are moving toward Industry 4.0, where machines are connected, processes run on data, and new digital tools are introduced continuously. Employees have to keep learning simply to stay current with the systems they work on. The same pace creates pressure on the companies themselves. The working environment has become more complex and less predictable, new technologies disrupt established ways of working, and skilled workers are in short supply [2]. At the same time many experienced employees are reaching retirement, and a large part of what they know leaves with them because it was never written down [3], [4]. The combined effect is information overload. In large organisations that operate across many domains, employees face a constant flow of documentation and training material, and finding the part of it that is both relevant and trustworthy becomes increasingly difficult [5].

In response, companies have begun to move from traditional learning toward AI guided learning. Large language models made this practical, since they can interpret and produce natural language well enough to answer questions in a conversational way. Their answers, however, are not always correct. Models produce confident but false statements, commonly called hallucinations, particularly when the training data is incomplete, outdated, or does not cover a specific or private domain [6]. For corporate training this is a serious weakness, because employees act on the answers they receive. Retrieval-Augmented Generation (RAG) was introduced to reduce this risk by bringing external information into the answering process. A RAG system first retrieves relevant material from an external source and then uses that material as context for the generated answer, which keeps the response closer to documented facts [7].

Bosch Rexroth Academy followed this approach and developed an AI supported learning assistant called AVAILABLE [8]. The system allows employees to learn directly from the company's own training material. Rather than searching through documents themselves,

users ask a question and receive a short answer together with references and links to the sources it came from. Each source carries a trustworthiness rating, so the user can judge how reliable the information is. In this way the system addresses not only the answer but also the path back to a reliable source. It works on the Academy’s real training material, which exists in several formats such as presentation slides, PDF documents and instructional videos. The system offers a question and answer mode, in which the user asks directly and receives a short answer with linked sources and their trust rating. It also offers a guided assistant mode, which establishes the user’s knowledge level through a few follow-up questions and then recommends learning content matching that level. A third feature supports knowledge capture through guided interviews, where an AI driven workflow helps collect knowledge from experts or retiring employees so that it can be stored and preserved. The design of this first version is described in the theory chapter, and the changes made to it in this thesis are described in the methodology.

The first version of the system carried several limitations, some of which the original work identified as open research questions [8]. The most critical one concerns trust assessment. Source trustworthiness was set manually through fixed heuristic rules, an approach that does not scale as the corpus grows and cannot resolve cases where two sources state contradictory information. Retrieval was restricted to semantic vector search, leaving no complementary mechanism to connect related content when the wording of the question and the wording of the document differ. The pipeline offered no observability into how retrieval and generation behaved at runtime, and no systematic evaluation procedure existed to quantify how well the system performed.

This thesis extends that baseline into a system referred to as AVAILABLE 2.0 and makes four contributions.

1. A systematic evaluation of the baseline to locate its performance and reliability weaknesses.
2. A hybrid retrieval mechanism that combines vector search with graph based search, integrated with the Opik observability tool so that the pipeline becomes transparent.
3. An automated trust scoring model that derives document trustworthiness from the documents themselves and replaces the manual heuristic.
4. A full evaluation using the RAGAS framework [9] to quantify the effect of these changes.

The main question this thesis answers is how the AVAILABLE learning assistant can be improved, through better retrieval and an automated trust model, so that knowledge

transfer in an industrial setting becomes more reliable and more trustworthy. The question is broken into three parts.

1. What are the main performance and reliability weaknesses of the baseline system, and how can they be identified systematically?
2. How can an automated trust scoring model be designed and placed into the pipeline so that it judges source reliability and replaces the manual approach?
3. How much do the implemented improvements raise retrieval and answer quality compared to the baseline?

The work begins with an exploration of the available data. The training material is spread across several databases, so an ETL pipeline is built to collect and organise it, which also makes the rest of the system more modular. The weaknesses of the current system are then identified, and several modern retrieval approaches are implemented to address them, together with the automated trust score and the observability layer. The outcome is a system of higher overall quality that scales better as the volume of incoming material grows.

The thesis first fixes the boundaries of the work and reviews the published research on retrieval-augmented generation, graph based retrieval and source reliability, which positions the contribution against what already exists. It then sets out the theoretical background needed later, covering language models, the baseline system, retrieval methods, trust scoring and the metrics used to judge retrieval and generation quality. The methodology follows and describes the architecture of AVAILABLE 2.0, the data preparation pipeline, the retrieval implementation, the automated trust scoring model and the observability layer. The evaluation then introduces the datasets and the experimental setup before reporting results for retrieval, generation and trust scoring. The findings are discussed in terms of what they mean for industrial knowledge transfer and where the approach reaches its limits, and the thesis closes with the conclusions drawn and the directions that remain open for future work.

2 Scope

This thesis focuses on the retrieval and trust components of the existing AVAILABLE system. It does not build a new chatbot from scratch. The main aim is to improve the underlying RAG pipeline by testing modern retrieval approaches and checking whether they lead to measurable improvements over the baseline, AVAILABLE 1.0. The pipeline is built with the LangChain framework¹. A hybrid retrieval method is implemented that combines vector search with graph-based search on a Neo4j² graph database. It is complemented by a cross-encoder reranker and the Opik³ observability tool. The results are compared against the baseline using the RAGAS⁴ evaluation framework [9].

The second focus of this thesis is the design and implementation of an automated trust scoring framework. Instead of the manual trust ratings used in the baseline, a document-level trust score is computed directly from the documents and their metadata. The framework is evaluated on the existing Bosch Rexroth training data to examine how well the automated scores reflect the actual reliability of the sources.

Some parts of the overall system are intentionally left outside the scope of this work. The multi-agent components, which are the question and answer mode, the guided assistant and the interview feature, are not modified, since the work targets the RAG pipeline and not the agent logic. This thesis also does not compare different large language models and does not train any model on the training data. The base language model is used as provided, and the work concentrates on the retrieval and trust layers around it.

¹<https://www.langchain.com>

²<https://neo4j.com>

³<https://github.com/comet-ml/opik>

⁴<https://docs.ragas.io>

3 Literature Review

Building a reliable enterprise learning assistant depends on the retrieval techniques that decide what information the system sees before it answers, and on whether that information can be trusted. This chapter reviews the research behind those techniques. It moves from the foundations of Retrieval-Augmented Generation, through the retrieval methods that find the right content, to the trust and evaluation methods that decide whether the content can be relied on and whether the whole system works.

3.1 Foundations of Retrieval-Augmented Generation

The starting point is Retrieval-Augmented Generation (RAG), a method that lets a language model pull relevant text from an external knowledge base before answering, so that its responses are grounded in real documents and not only in what the model memorised during training [10]. How well this works depends on the retrieval step. If the right documents are not found, even a strong language model cannot produce a correct answer. Early RAG systems relied on dense vector retrieval, where the query and the documents are turned into embeddings and matched by similarity, usually cosine distance. This captures meaning well, so a question can match a passage even when the two share no exact words. Dense retrieval has a known weakness, however. It often struggles with exact keyword matching, out-of-domain terms and domain-specific acronyms, which are common in technical and enterprise settings [6]. Hybrid search addresses this by combining dense retrieval with sparse, keyword-based retrieval, so that the system benefits from both semantic understanding and exact lexical matching. This combination has been shown to improve both recall and precision [11].

3.2 Retrieval with Embeddings, Graphs and Reranking

Achieving this hybrid combination in practice depends on the embedding model, since it turns both queries and documents into the vectors used for retrieval. Most embedding models are built for a single type of retrieval, but some support several at once. BGE-M3 is one such model. Within a single architecture it produces dense, sparse and multi-vector representations of the same text, so it can serve semantic and keyword-based retrieval together. Through a training technique called self-knowledge distillation, it also generalises better across complex retrieval tasks than traditional dense-only models [12].

Vector search still treats each chunk of text as an independent point and matches it by meaning alone. It ignores the connections between pieces of text and the structure of the documents they come from. For hierarchical or networked information this is a real limitation, and it has motivated work that combines retrieval with explicit graph structures. In such systems vector search finds the broad meaning, while the graph captures exact connections between entities.

Some methods use graphs to capture the structure of a whole corpus. GraphRAG builds a knowledge graph from the documents, groups related entities into communities and summarises each one, which helps answer broad questions that plain retrieval handles poorly [13]. RAPTOR follows a similar idea but organises the information as a tree of summaries, so that the system can retrieve at different levels of detail [14]. Both methods first find the right region of the corpus and then retrieve the specific content within it.

Other methods focus on spreading relevance across the graph. A common starting point is multi-hop entity traversal, where the system identifies the entities mentioned in the query and explores their neighbourhood in the graph. By following edges outward it reaches content that is connected to the query but does not match its wording directly [15]. For this the system first needs to know which entities the query is about. A common way is to embed every entity, store the embeddings in a vector index, embed the query with the same model, and select the most similar entities as starting points [16]. From these seed entities, relevance can be propagated more formally with PageRank, a classic algorithm that ranks nodes by how a random walk moves through a network [17], and with its personalised variant, which pulls the walk toward the chosen starting nodes [18]. HippoRAG builds on this idea. It runs Personalized PageRank from the entities of the query and gives more weight to rare, specific terms than to common ones [16]. HippoRAG 2 goes further by also starting the walk from passages found through dense retrieval, which links the graph more closely to vector search [19].

A well-known example of combining both retrieval types is the HybridRAG framework. It extracts relational triplets, made of a subject, an action and an object, from documents to build a knowledge graph alongside the vector database. In experiments on financial earnings call transcripts it improved both retrieval accuracy and answer generation over traditional RAG [20]. Once passages have been retrieved from both the vector store and the graph, the combined set still needs to be ordered well before it reaches the language model. Passing a long, unordered list of documents causes a known problem, because language models tend to ignore information placed in the middle of a long context

and focus on the beginning and the end [21]. To avoid this, an intermediate reranking step is used, typically with a cross-encoder model. The first retrieval stage scores each document independently of the query, whereas a cross-encoder reads the query and a document together and judges how well they match [22]. This pushes the most relevant passages to the top, so the language model receives clean, well-ordered context instead of a noisy list.

Across these graph methods, most systems use either a semantic graph of entities or a structural graph of the document hierarchy, but rarely both together. They are also usually tested on multi-hop questions, where the answer is spread across several documents, and not on the single-hop lookups that are common in enterprise documentation. This leaves open how graph retrieval performs when both signals are combined and tested on a real enterprise knowledge base, which is one of the focus areas of this thesis.

3.3 Trust and Reliability of Source Documents

Good retrieval finds relevant documents, but relevance alone does not guarantee that a document is reliable. RAG improves answers by grounding them in retrieved documents, yet it quietly assumes that those documents are trustworthy in the first place. Even a system that cites its sources perfectly can be wrong if the sources are of poor quality, because the model can only reason over what it is given [23]. This makes the trustworthiness of the source collection a concern that comes before retrieval and not after it. If unreliable documents sit in the knowledge base, they can be retrieved and passed to the model just as easily as good ones.

A long line of work treats trust not as a property judged for each document in isolation, but as something that flows through a graph of connected documents. The classic example is TrustRank, which starts from a small seed of known good pages and propagates trust across the link graph of the web. Pages linked to trusted ones are treated as more trustworthy, while likely spam is pushed down [23]. TrustRank builds directly on PageRank and applies the same propagation idea to reliability instead of popularity.

More recent work has brought this propagation idea into RAG document collections. ClaimTrust adapts a PageRank-style algorithm to score documents by whether the factual claims they contain support or contradict one another. Documents corroborated by others gain trust, while isolated or contradicted ones lose it [24]. RAGRank follows a similar direction but focuses on the placement in the pipeline. It computes a PageRank-

based authority score for each document and uses it together with the retrieval similarity score as a second filtering step, so that low-credibility documents are less likely to reach the model [25]. Both show that graph-based trust propagation can rank source documents by reliability without heavy manual labelling.

Trust is rarely captured by a single signal. Work in high-stakes technical domains argues that document credibility is best assessed from several complementary perspectives at once. These are structural trust from how documents connect in a graph, source trust from the author and institution behind a document, and content trust from intrinsic quality indicators [26]. Combining these signals gives a more stable picture than graph structure alone, especially when the graph is sparse or a document has few connections. Taken together, these methods show that source trust can be estimated automatically and propagated through a document graph. Most existing work, however, is built around web spam or news and claim verification, and treats trust either as pure graph propagation or as a single-source judgement. Less attention has been paid to computing a document-level trust score before retrieval that combines a metadata-based prior with graph propagation, and to doing so for a real enterprise knowledge base. This is the setting this thesis addresses.

3.4 Evaluation and Observability of RAG Systems

Once a system retrieves and answers, its quality has to be measured. Evaluating a RAG system is not as simple as checking whether the final answer is correct, because a RAG pipeline has two separate parts that can each fail on their own. The retriever might pull the wrong documents, or the generator might write an unsupported answer even when the right documents were retrieved. For this reason most research evaluates the two parts separately. The retrieval step is judged on whether it found the right content, and the generation step is judged on whether the answer is faithful to that content and relevant to the question [27]. This separation makes it much easier to see where a problem comes from.

On the retrieval side, a few standard metrics are commonly used. Hit@k checks whether at least one correct chunk appears in the top k results. Recall@k goes further and measures how many of the existing correct chunks were found, which shows how much was missed. Mean Reciprocal Rank (MRR) looks at where the first correct chunk lands in the ranked list, so a system that places the right answer near the top scores higher

than one that buries it lower down [28]. Together these show whether something correct was found at all, how much was found, and how well it was ranked.

On the generation side, the focus shifts to the quality of the answer itself, mainly whether it is faithful to the retrieved content and whether it addresses the question. Checking this by hand for every answer does not scale, so frameworks have been built to automate it. RAGAS is a widely used framework that scores a RAG system on a fixed set of metrics such as faithfulness, answer relevance, and context precision and recall. It uses an LLM to compute them and needs only a small amount of ground truth [9]. This makes it practical to evaluate many questions quickly and consistently.

Some parts of answer quality are hard to capture with fixed formulas, for example whether an answer sounds natural or fully matches the intent behind a question. To handle this, researchers use an LLM itself as the evaluator and prompt it to score a response the way a human reviewer would. This approach, known as LLM-as-a-judge, agrees with human preferences in more than 80% of cases when a strong LLM does the judging [29]. It is not perfect and can carry biases, such as favouring longer answers or being influenced by the order in which responses are shown, but it remains a practical way to evaluate answer quality at scale.

Metrics show how well a system performs, but not what happened inside it when something goes wrong. As RAG pipelines grow more complex, checking whether the final answer is right is no longer enough. Research has argued for a shift away from black-box testing toward deep system observability, which tracks what happens at every internal step, including retrieval paths, token costs, latency and errors [30], [31]. Because language models are non-deterministic, the same question can take a different internal path on two runs, which makes standard debugging tools poorly suited to finding the root cause of a problem. Keeping a detailed record of every action as traces and spans, from the initial retrieval search through to the final reranking, addresses this. Structured trace logs have been shown to let developers diagnose root causes, find where the system slows down, and use that evidence to reduce cost and improve the overall design [30].

3.5 Research Gap

Across these three areas a clear pattern emerges. Retrieval research has moved from single dense vector search toward hybrid and graph-based methods, but most of this work is tested on general or multi-hop benchmarks and not on a real, single-domain enterprise knowledge base, and few systems combine semantic and structural graph signals. Trust

research has shown that reliability can be propagated through a document graph instead of being judged document by document, but this idea has mainly been applied to web content and claim verification, and rarely to a document-level score computed before retrieval in an enterprise setting. Evaluation research provides solid metrics for judging retrieval and generation separately, and observability tooling makes it possible to see why a pipeline produced a given answer, but these tools are rarely applied together to compare a baseline system against an improved one in a systematic way.

These three gaps are not separate problems. A retrieval system that finds more relevant content is only useful if the sources behind that content can also be trusted, and both improvements can only be judged reliably with proper evaluation and observability in place. This thesis addresses all three together. It extends the AVAILABLE learning assistant with hybrid retrieval, an automated trust score, and a full evaluation and observability setup, so that the three sub-questions introduced in the introduction can be answered as one connected piece of work and not in isolation.

4 Theory

4.1 LLM and RAG Background

Large Language Models (LLMs) are neural networks trained on very large amounts of text to predict the next word in a sentence. They are built on the Transformer architecture, which processes text as a sequence of tokens and uses a mechanism called self-attention to weight how strongly each token influences every other token [32]. During training, the model is repeatedly given a piece of text with the next word hidden, and it learns to predict that word by adjusting billions of internal parameters, also called weights. Once training is finished, this knowledge is fixed. The model has no separate memory and can only generate text from the patterns already stored in its parameters.

This creates a problem for company use. An LLM is not trained on the private documents of a company, such as internal machine manuals or training material. When it is asked about something outside its training data, the model often produces an answer that is fluent and confident but factually wrong. This behaviour is called hallucination⁵ [6].

Retrieval-Augmented Generation (RAG) was introduced to address this problem [10]. Instead of relying only on what the model has memorised, a RAG system first searches an external knowledge base for passages that are relevant to the question and then passes those passages to the LLM as context. The LLM is instructed to base its answer on this retrieved context and not on its own parameters.

A basic RAG pipeline consists of three stages that build on one another. The first stage is indexing. The knowledge base, in this case the Bosch Rexroth training documents, is split into smaller pieces ahead of time and stored in a form that can be searched efficiently. When a user question arrives, the retrieval stage searches this index and returns the pieces judged most relevant to the question. In the generation stage, the LLM receives the original question together with the retrieved pieces and produces an answer grounded in them.

This basic setup raises two further questions that the rest of this chapter addresses. The first is how the retrieval stage should decide what counts as relevant. The second is how the system should judge whether a retrieved document can be trusted in the first place.

⁵need to add

4.2 Retrieval Methods

4.2.1 Vector Retrieval

Vector retrieval works by turning text into a numerical form that captures meaning, so that pieces of text with similar meaning end up close together in a shared space. This numerical form is called an embedding, a fixed-length list of numbers produced by an embedding model, such that texts about the same topic map to nearby points even when they use different words. This is what lets vector retrieval match a question to a passage without the two sharing the same vocabulary, a property that earlier keyword-based search methods lacked [33].

At index time, every chunk of the knowledge base is passed through an embedding model and the resulting vector is stored in a vector database. At query time, the user's question is embedded in the same way, and the database returns the chunks whose vectors are closest to the question's vector, usually measured by cosine similarity⁶.

This meaning-based approach, known as dense retrieval, has a known weakness. Because it matches on overall meaning, it can blur the difference between two texts that are semantically similar but differ in a specific detail, such as an exact code, a part number or a technical term [6]. To address this, dense retrieval is often paired with sparse retrieval, an older approach that scores text by exact word overlap, weighted by how rare or informative each word is [34]. Sparse retrieval is weaker at capturing meaning but stronger at catching the exact terms that dense retrieval can miss.

Modern embedding models such as BGE-M3 combine both signals. From the same input and in a single pass they produce a dense vector for meaning and a sparse vector for exact terms, rather than needing two separate models [12]. When both result lists are available, they are commonly merged using reciprocal rank fusion, a method that combines rankings based on each item's position in each list rather than its raw score, which avoids comparing two differently scaled similarity scores directly [35].

4.2.2 Graph Retrieval

Vector retrieval treats every chunk as an independent piece of text and scores it in isolation. This is a limitation when the answer to a question is not contained in a single passage but depends on how two separate pieces of information are connected. A knowledge graph addresses this by representing information as a set of nodes connected by edges. Nodes typically stand for entities, such as a part name, a process or a

⁶need to add

document, and edges represent the relations between them [36]. Because relations are stored explicitly, a graph can link two passages that discuss the same underlying concept even when they share little vocabulary, which is exactly the case dense vector search tends to miss.

Building a graph for retrieval usually involves an extraction step. A language model reads the text and identifies the entities it mentions and the relations between them, which are then merged so that the same entity mentioned in different places becomes a single node [13]. Each chunk is linked to the entities it mentions, so a query can be matched to a small set of relevant entities first and then extended outward to the chunks connected to them.

Not every entity is equally informative. An entity that appears in almost every document, such as a generic term, says little about what makes a specific question distinctive, while a rare, specific entity narrows things down far more. This is captured by inverse document frequency (IDF), which assigns higher weight to entities that occur in fewer documents [37]. Query entities are therefore weighted by a combination of how confidently they were matched and how specific they are, so that rare, precise concepts have more influence on retrieval than common ones.

Once the graph is built, there are different ways to spread relevance from a query's matched entities to the chunks that should be retrieved. The simplest approach, entity hop, follows the direct edges from a matched entity to the chunks that mention it, and optionally repeats this for a fixed number of hops. This works well when the answer sits close to the matched entity, but it treats each step as an isolated jump and does not consider how well connected a chunk is within the graph as a whole.

A more general approach is Personalized PageRank (PPR), which extends the original PageRank algorithm used to rank web pages by their incoming links [17]. Instead of measuring importance in a generic, query-independent way, PPR restarts the random walk at a fixed set of seed nodes, so that importance is measured relative to a specific starting point rather than the whole graph [18]. Formally this is expressed as

$$\pi = \alpha \cdot S \cdot \pi + (1 - \alpha) \cdot e \quad (4.1)$$

where π is the importance score of each node, S is the graph's transition matrix, which describes how weight passes from one node to its neighbours, e is the starting vector that places weight only on the seed nodes, and α is the damping factor, which controls how far the walk is allowed to spread before it is pulled back to the seeds [17]. A higher

α lets relevance travel further across the graph, while a lower α keeps it concentrated near the original query entities. This restart mechanism is what lets PPR reach chunks that are several relations away from a seed entity, a case that plain entity hop tends to miss [16].

4.2.3 Hybrid Retrieval

Vector retrieval and graph retrieval each capture a different kind of relevance. Vector retrieval is strong at matching a question to a passage that expresses the same idea in different words, while graph retrieval is strong at connecting passages through shared entities and relations, even when the wording is completely different. Because the two approaches fail in different situations, combining them tends to recover more relevant passages than either one alone [20]. This combination is generally called hybrid retrieval.

The most direct way to combine two retrieval legs is a union. The top results from each leg are merged into a single candidate pool, with duplicates removed. A passage retrieved by both legs is naturally a strong candidate, since two different retrieval signals agree on it independently. This union step does not try to produce a final ranking by itself. It only decides which passages are worth considering further and leaves the actual ordering to a later step.

This later step is needed because the scores from the first-stage retrieval are not directly comparable and, on their own, are only a rough estimate of relevance. Vector similarity and graph traversal scores are computed independently and sit on different scales, and both are produced by comparing the query and each passage separately, without the two ever being read together. A more precise way to judge relevance is a cross-encoder. Instead of embedding the query and a passage separately and comparing their vectors, a cross-encoder takes the query and one candidate passage together as a single input and directly predicts how well that passage answers the query [38]. This joint comparison is considerably more accurate than similarity search, but it is also far more expensive, since it must run once for every query-passage pair rather than once per passage in advance. For this reason cross-encoders are used only as a second, reranking stage, applied to the small candidate pool returned by the first-stage retrieval rather than to the whole corpus [22].

Reranking with a cross-encoder also helps counter a separate problem in long-context generation. Language models do not use all parts of a long context equally well and tend to pay less attention to information placed in the middle than to information near the beginning or the end, a pattern known as the lost in the middle effect [21]. By reordering

the retrieved passages so that the most relevant ones come first, reranking reduces the chance that an important passage is overlooked because of where it happened to fall in the context window.

4.3 Trust Score

Retrieval, as covered in the previous section, is built entirely around relevance. It asks how closely a passage matches the meaning or the entities of a question. Relevance alone, however, says nothing about whether a passage can be trusted. Two documents can be equally relevant to a question while differing sharply in reliability. One might be a current, expert-authored handbook, and the other an outdated draft from an unknown author. A retrieval system that only measures relevance cannot tell these apart, which becomes a real problem in enterprise knowledge bases, where documents accumulate over time, some maintained carefully and others left unreviewed for years.

This gap has led to two broad ways of estimating how trustworthy a document is.

The first looks at the document on its own, independent of anything else in the corpus. This content-based view judges a document by properties such as who authored it, how recently it was updated, how complete and specific its content is, and what format it is stored in [26]. These signals are cheap to compute, since each document can be scored without reference to any other document, but they only capture what can be observed from the document in isolation.

The second view judges a document by how it relates to the rest of the corpus, an idea that originates from web search. Early search engines faced a similar problem with spam. A page could rank highly by relevance while containing unreliable or manipulated content. TrustRank was introduced to address this by propagating trust through hyperlinks rather than judging every page in isolation [23]. A small set of pages is first labelled as trustworthy by a human reviewer, and trust then spreads outward along links using the same mathematical process as PageRank, so that a page linked to by many trustworthy pages accumulates higher trust than a page with no such connections [17]. The underlying assumption is that trustworthy sources tend to reference other trustworthy sources, so corroboration itself carries information about reliability.

This idea has since been adapted from web pages to documents in a retrieval corpus. Instead of hyperlinks, a document-to-document graph is built from shared content, such as entities or claims that appear in more than one document, and trust is propagated across this graph in the same way it propagates across links on the web. ClaimTrust

applies this by checking whether claims in one document are supported or contradicted by claims in others, which allows trust to rise when information is independently corroborated [24]. RAGRank instead combines a document’s authority with its retrieval similarity, so that documents with lower credibility are down-weighted before they are selected [25]. Both approaches share the logic of TrustRank. Trust is not only a property of a single document, but something that can flow from reliable sources to the documents connected to them.

In practice, content-based and propagation-based signals are complementary rather than competing. A document with strong content and metadata but no connections to the rest of the corpus still deserves a reasonable trust estimate, while a document that is well corroborated by other trustworthy sources provides evidence beyond what its own content reveals. Combining the two into a single score, typically by using the content-based estimate as a starting point and adjusting it through propagation across the corpus, lets a trust score be computed automatically for every document without relying on manual review [26].

4.4 Evaluation Metrics

Building an improved retrieval and trust system is only useful if its effect can be measured. Evaluation in a RAG pipeline is generally split into two separate questions, because retrieval and generation can each fail independently of the other [27]. The first question is whether the system found the right document at all. This is a retrieval question, and it can be checked automatically once the correct source for a question is known in advance. The second question is whether the answer written from that document is actually good. This is a generation question, and it is harder to check automatically, since the same fact can be phrased correctly in many different ways. This section introduces the metrics used for each of these two questions in turn.

4.4.1 Retrieval Evaluation Metrics

Retrieval metrics assume that, for a given question, one or more documents are already known to be the correct source, commonly called the golden document. Retrieval is then evaluated by checking where this golden document falls in the ranked list the system returns, rather than by judging the retrieved text itself [28].

The simplest such metric is Hit@ k , which checks whether the golden document appears anywhere within the top k results returned for a question.

$$\text{Hit@}k = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[r_i \leq k] \quad (4.2)$$

Here r_i is the rank at which the golden document was found for question i , N is the number of questions, and the indicator function returns 1 if the golden document falls within the top k and 0 otherwise [28]. Hit@ k is usually reported at more than one cutoff, since a small k , for example Hit@1, tests strict precision at the very top of the ranking, while a larger k , for example Hit@10, tests whether the document is found anywhere within a wider window. The wider window matters more when several passages are passed on to generation rather than just one.

Hit@ k treats every rank within the cutoff equally. Whether the golden document appears at rank 1 or rank 10, Hit@10 scores both cases the same. This can hide a meaningful difference, since a document ranked first is more useful to a downstream reranker or generator than one buried near the bottom of the window. Mean Reciprocal Rank (MRR) addresses this by rewarding a higher position directly.

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{r_i} \quad (4.3)$$

A golden document found at rank 1 contributes a full point, one found at rank 2 contributes one half, and a question for which the golden document is never retrieved contributes nothing [28]. MRR is therefore a stricter measure than Hit@ k . Two systems can have identical Hit@10 scores while differing considerably in MRR, if one of them consistently ranks the correct document closer to the top.

A related measure is Recall@ k , which is close to Hit@ k but generalises it to the case where a question may have more than one correct document. It measures what fraction of all correct documents were found within the top k , rather than just whether one was found. Together, Hit@ k , MRR and Recall@ k give a view of retrieval quality from two angles, whether the correct evidence is found at all, and how well it is ranked once it is found.

4.4.2 Generator Evaluation Metrics

Retrieval metrics can confirm that the correct document was found, but they say nothing about the text the language model produces from it. A model can be given the correct passage and still write an answer that ignores it, misreads it, or adds unsupported details, so generation has to be evaluated separately, on the answer itself rather than on the ranking that produced its context [27]. Because a correct answer can be phrased in

many different ways, it cannot be checked by simple string matching against a reference answer the way a golden document can. The common solution is to use another language model as a judge, an approach shown to align reasonably well with human evaluation on open-ended text [29]. RAGAS is a framework built specifically for RAG pipelines that uses this idea to score both the answer and the context it was built from, rather than treating generation as a single, all-or-nothing outcome [9].

RAGAS defines four metrics, two that judge the retrieved context and two that judge the generated answer.

Context precision measures how much of the retrieved context is relevant, rewarding a system that places the useful passages near the top of the context rather than mixed in with irrelevant ones.

$$\text{Context precision} = \frac{\sum_k \text{Precision@k} \cdot v_k}{\text{number of relevant contexts}} \quad (4.4)$$

Here v_k is 1 if the passage at rank k is relevant and 0 otherwise [9]. A low score here signals that the retrieved context is noisy, even if the correct passage is present somewhere within it.

Context recall measures whether the retrieved context contains what is needed to answer the question. The reference answer is broken down into individual claims, and each claim is checked for support in the retrieved context.

$$\text{Context recall} = \frac{\text{ground-truth claims supported by the context}}{\text{total claims in the ground truth}} \quad (4.5)$$

A low score here means part of the needed information was missing from retrieval altogether, regardless of how well the model writes.

Faithfulness checks whether the generated answer stays grounded in the retrieved context, rather than adding information the context does not support. The answer is broken down into individual claims, and each is checked against the context.

$$\text{Faithfulness} = \frac{\text{claims in the answer supported by the context}}{\text{total claims in the answer}} \quad (4.6)$$

This is effectively a hallucination check applied directly to generation. A faithful answer sticks to what the retrieved passages actually say, even if that happens to be incomplete.

Answer relevancy checks whether the answer addresses the question that was asked, independent of whether it is faithful to the context. This is measured indirectly. The

judge model generates a small set of questions that the answer appears to be responding to, and each generated question is compared to the original question by embedding similarity.

$$\text{Answer relevancy} = \frac{1}{n} \sum_{i=1}^n \cos(q, q_i) \quad (4.7)$$

Here q is the original question and q_i are the questions generated back from the answer [9]. An answer that drifts off topic produces generated questions that diverge from the original, which lowers the score even if the answer is technically faithful to the context. Used together, these four metrics separate two failure modes that a single overall score would blur together. A low context score points to a retrieval problem, while a low faithfulness or relevancy score points to a generation problem, even when the correct context was retrieved.

4.5 Observability

A RAG pipeline performs several steps for every single question. It embeds the query, searches one or more retrieval legs, reranks candidates, folds in additional signals such as trust, and finally calls the LLM to generate an answer. When a poor answer comes out at the end, reading that final answer alone does not reveal which of these internal steps caused the problem. The question may have been misembedded, the correct document may never have been retrieved, or the document may have been retrieved but the LLM failed to use it correctly. Without visibility into what happened at each step, debugging a pipeline like this is largely guesswork.

Observability refers to the practice of instrumenting a system so that its internal behaviour can be inspected after it runs, rather than only its final output [31]. In traditional software systems this is often done through logging, but a multi-step AI pipeline benefits from a more structured form of tracing, since it needs to preserve not only that a step happened, but also what data flowed into and out of it, how long it took, what it cost, and how the steps are nested within one another [30].

This structure is commonly organised into three levels. A span represents a single step in the pipeline, such as one call to the vector store or one call to the language model, and records that step's inputs, outputs and duration. A trace represents one complete request from end to end, made up of all the spans it triggered, arranged in the order and nesting in which they occurred, so that a single trace shows the full path a question took through the system. A thread groups multiple traces that belong to the same session,

which matters for systems used conversationally, where a person may ask several related questions in a row [30].

Recording spans and traces automatically, rather than inserting log statements by hand at every step, is typically done by wrapping each function of interest so that its execution is captured without changing how the function itself works. When one such wrapped function calls another, the resulting spans are nested automatically, which reconstructs the shape of the pipeline as a tree without any of this structure being built by hand.

Beyond recording what happened, observability tooling can also attach automated quality feedback to each trace. An LLM-based judge, of the kind introduced in the generator evaluation metrics above, can score a completed trace for properties such as hallucination or context relevance and attach that score directly to the trace it evaluates, so that quality signals and the technical details of a request are available together rather than in separate systems.

For a pipeline built from multiple retrieval legs, a reranker and a trust-scoring step, this kind of tracing is what makes it possible to tell, for any single question, exactly where in the pipeline an unexpected result originated.

4.6 AVAILABLE 1.0

The system extended in this thesis, AVAILABLE, was developed by the Bosch Rexroth Academy as an AI-supported learning assistant for on-the-job training [8]. Its purpose is to help employees find and apply relevant company knowledge without searching manually through scattered documentation, course catalogs or training portals. The original authors report that such a search can otherwise take several minutes per query and often returns incomplete results.

AVAILABLE 1.0 offers two modes of interaction, both built on a Retrieval-Augmented Generation (RAG) approach. The first is a question and answer mode, where a user asks a question directly and receives a short answer together with references and links back to the original source material. The second is a guided assistant mode, which first identifies the learning objective and current knowledge level of the user through a short dialogue of follow-up questions, and then recommends training content suited to that level.

The guided assistant mode is implemented as a multi-agent system, coordinated with the AutoGen framework, with GPT-4o as the underlying language model served through Azure. Three agents divide the work. A Topic Agent determines what the user wants to learn from their responses. A Knowledge Agent classifies the expertise of the user into

one of three levels, beginner, advanced or expert, through didactic follow-up questions. A Curriculum Agent combines both outputs to search for and present suitable content, together with a reliability rating for each source. Underneath both modes sits a shared document processing pipeline that generates hierarchical summaries of the training material. It links specific passages to their broader document context, so that retrieval can return not only a matching passage but also the surrounding context needed to interpret it.

Document trustworthiness in this baseline is assessed only heuristically. The original authors describe the rating as an initial, algorithmically generated estimate that has not been formally validated. For this reason the system was populated only with material already known to be reliable, and it was not tested against a mixed corpus of varying quality. A pilot evaluation on 70 training documents and 60 videos showed clear efficiency gains over manual search. The time to find a relevant document dropped from several minutes to a few seconds, at an accuracy above 95% across the tested use cases [8].

The same evaluation leaves two questions open. The first is how the system should behave when it meets multiple, possibly contradictory sources. The second is how a reliable, automated measure of source trustworthiness could be derived in the first place [8]. These two open questions are the direct motivation for the retrieval and trust scoring work developed in this thesis.

5 Methodology

5.1 System Overview and Architecture

AVAILABLE is a learning assistant built on the training material of the Bosch Rexroth Academy. It supports several learning use cases, such as a guided assistant that recommends training material based on the topic and level of a user, and a question and answer mode that returns a direct answer to a question. These modes differ in how they interact with the user, but they share the same backbone, a Retrieval-Augmented Generation pipeline that searches the training corpus for relevant passages and then uses a large language model to write an answer grounded in them. The project exists in two generations. AVAILABLE 1.0 was developed by the Bosch team [8] and serves as the baseline of this thesis. AVAILABLE 2.0 is the system developed in this work, which extends the baseline. This section describes how AVAILABLE 2.0 is built and uses the first version as the reference point against which the changes are explained.

In AVAILABLE 1.0, retrieval relies on semantic search alone. During preparation, the training documents are converted to text, embedded and stored in a FAISS vector index. At query time the question is embedded in the same way and matched against the index by vector similarity. The closest passages are passed to GPT-4o, deployed on the Azure infrastructure of Bosch, which writes the answer and cites the training material it used. Document trust is handled manually. A training folder can contain an optional trust file in which the responsible team marks a document as unchecked, trusted, outdated or wrong. This flow is shown in Figure 5.1.

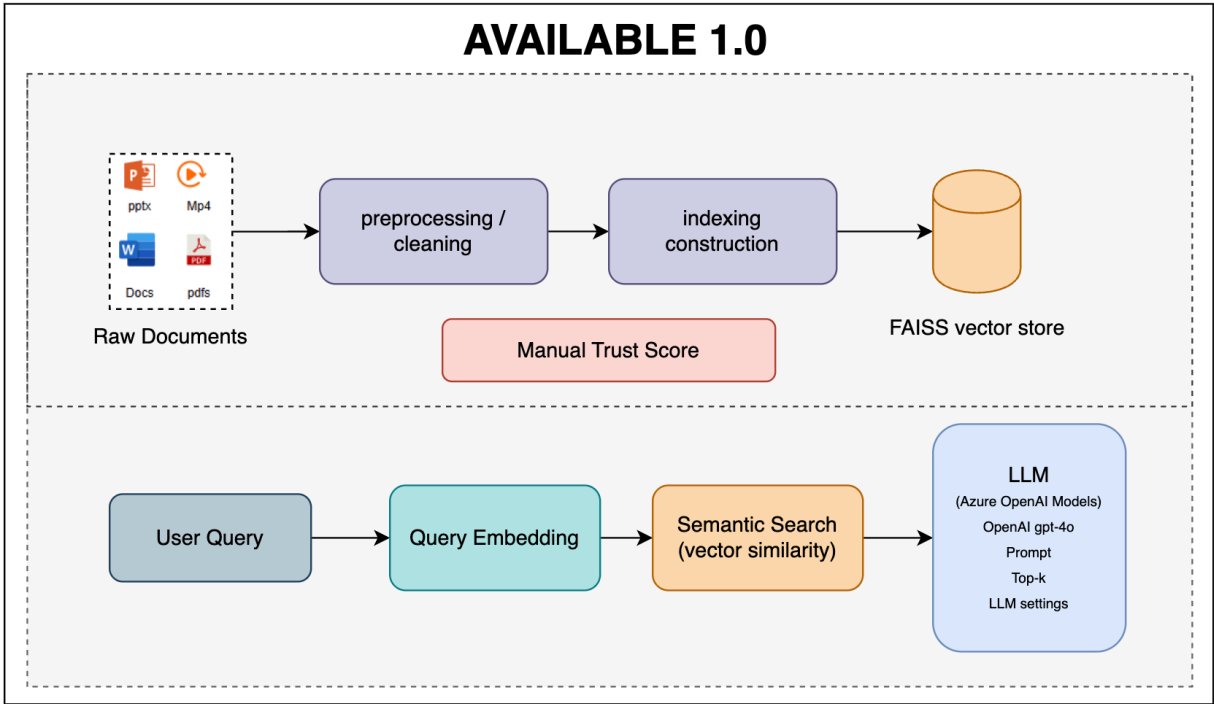


Figure 5.1: AVAILABLE 1.0 architecture, with document preparation above and query handling below.

The first version established the core idea and the interface of the system, but feedback from the internal teams pointed to several limitations. The sources retrieved for a question often did not match what was asked. Teams wanted to upload new training material themselves instead of depending on the development team. They also asked for visibility into the behaviour of the system in production, covering latency, cost, errors and recent questions, and for a way to compare a baseline against an improved system on defined metrics before any change goes live. A review of the system added one further finding. The trust of a document depended entirely on a manual rating that most documents never received. Table 5.1 links each of these points to the part of AVAILABLE 2.0 that addresses it.

Table 5.1: Comparison of AVAILABLE 1.0 and AVAILABLE 2.0.

Topic	AVAILABLE 1.0	AVAILABLE 2.0
Document intake	Added manually by the development team	Collected automatically from SharePoint sites
Retrieval	Semantic search in a FAISS index	Hybrid search over a vector store and a knowledge graph
Ranking	Order by vector similarity	Cross-encoder reranking with a trust boost
Document trust	Optional manual rating per folder	Automated trust score for every document
Monitoring	Not available	Tracing of every step with Opik
Evaluation	No systematic procedure	Evaluation harness with retrieval and generation metrics

AVAILABLE 2.0 addresses these points through new components placed around the same retrieval and generation core. The architecture is shown in Figure 5.2 and is divided into three pipelines.

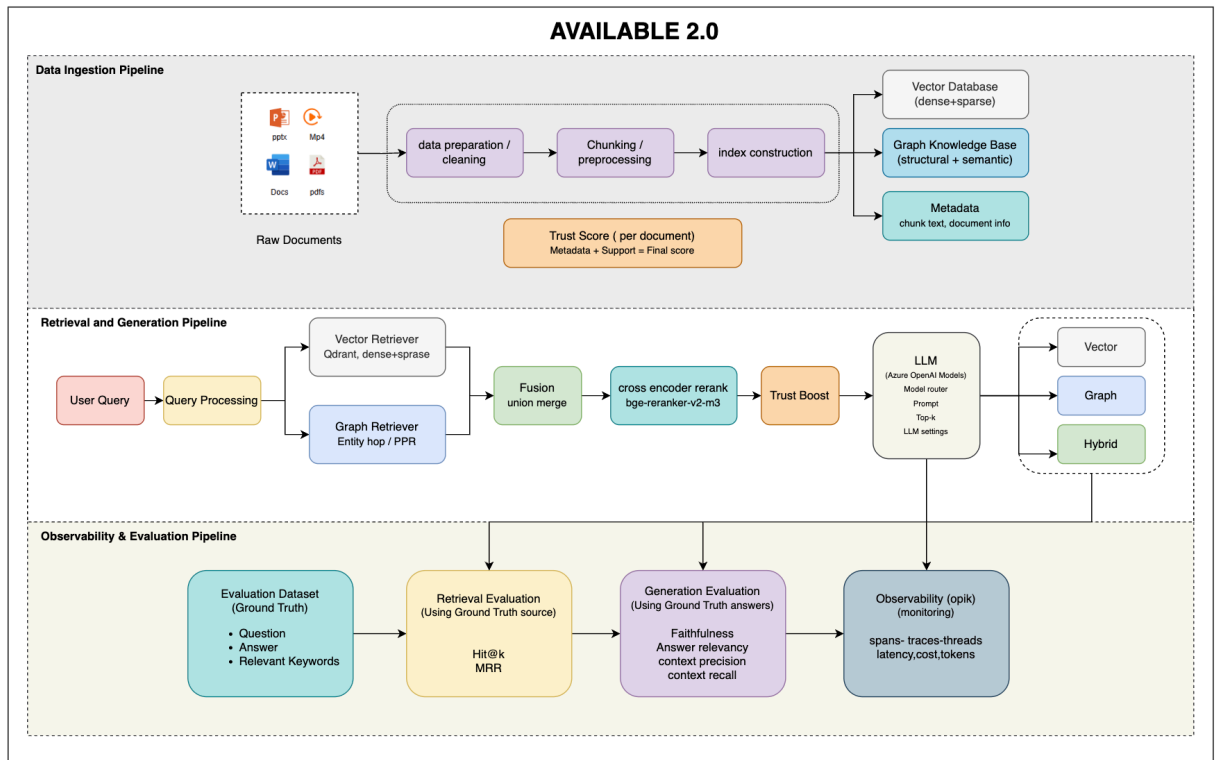


Figure 5.2: AVAILABLE 2.0 architecture with the data ingestion, retrieval and generation, and observability and evaluation pipelines.

The data ingestion pipeline runs ahead of any query and produces everything the query time flow depends on. Material is collected and cleaned from several SharePoint sites

into one organised data store, which removes the dependency on the development team for adding new content. The cleaned material is split into chunks. These chunks are used to build three stores at once, a vector database with dense and sparse representations for semantic search, a knowledge graph that connects chunks through the structure of the documents and through entities and relations extracted by an LLM, and a metadata store holding the chunk text and document information. The graph is what allows related content to be found when the wording of a question does not match the wording of a passage. A trust score is also computed for each document at this stage, built from document metadata and supporting evidence, and it replaces the manual rating.

The retrieval and generation pipeline handles a question at query time. The question is searched against the vector store and the knowledge graph in parallel, and the two result lists are merged by union into one candidate pool. A cross-encoder reranker then scores this pool more precisely, and the trust score of each document is applied at this step as a boost. More trustworthy documents move higher in the ranking, while less trustworthy ones are not removed from the pool. The top passages are passed to the LLM, which generates the final answer. The pipeline can run in vector, graph or hybrid mode, which makes it possible to compare the retrieval approaches under identical conditions.

The observability and evaluation pipeline sits beside the live flow. Every step of a request is traced with Opik, which provides the visibility into latency, cost, errors and recent questions that the teams asked for. An evaluation harness runs separately on its own dataset of questions with ground truth, and scores retrieval and generation with the metrics introduced in the theory chapter. This allows a baseline and an improved version to be compared before a change reaches production. The components of the three pipelines are described in detail in the sections that follow.

5.2 Data Preparation and Cleaning

The quality of a Retrieval-Augmented Generation system depends heavily on the quality of the data it retrieves from [10]. No downstream method can recover information that is missing at the source, out of date, or represented inconsistently. Gathering this material and keeping it in a consistent and current form is therefore a precondition for the retrieval methods that follow. The result is a structured corpus on which the retrieval components build.

5.2.1 Data Source

The corpus used in this thesis consists of 247 documents drawn from three topics, Grownfield, Digital Transformation and Hydraulics, and it spans both German and English material. Hydraulics covers engineering training on hydraulic systems, from fundamentals such as pressure and flow through to components and troubleshooting. Digital Transformation covers strategic and conceptual training on Industry 4.0 and the digitalisation of manufacturing. Grownfield, the largest topic by far, covers hands-on process training tied to a company-wide migration of business processes to a new SAP system. The distribution of documents across topics is shown in Table 5.2, and the distribution across file formats is shown in Table 5.3.

Table 5.2: Distribution of the corpus across topics.

Topic	Documents
Grownfield	TODO
Digital Transformation	TODO
Hydraulics	TODO
Total	247

Table 5.3: Distribution of the corpus across file formats.

File format	Documents
PowerPoint	TODO
PDF	TODO
Video	TODO
Other	TODO

5.2.2 Data Processing

The data preparation follows a medallion architecture⁷, a staged design in which raw input is refined through successive layers of increasing structure [CITE: Databricks medallion architecture]. Three layers are used in this work. The Bronze layer gathers the raw files from SharePoint and keeps a local copy synchronised with the source. The Silver layer reads each file and turns its content into uniform, structured records in one common representation. The Gold layer gathers these records into the chunk collection that the retrieval stage indexes. Figure 5.3 shows the full pipeline with the three layers marked.

⁷The medallion architecture is a data design pattern from the data lakehouse paradigm, in which data is refined across successive bronze, silver and gold layers of increasing quality and structure.

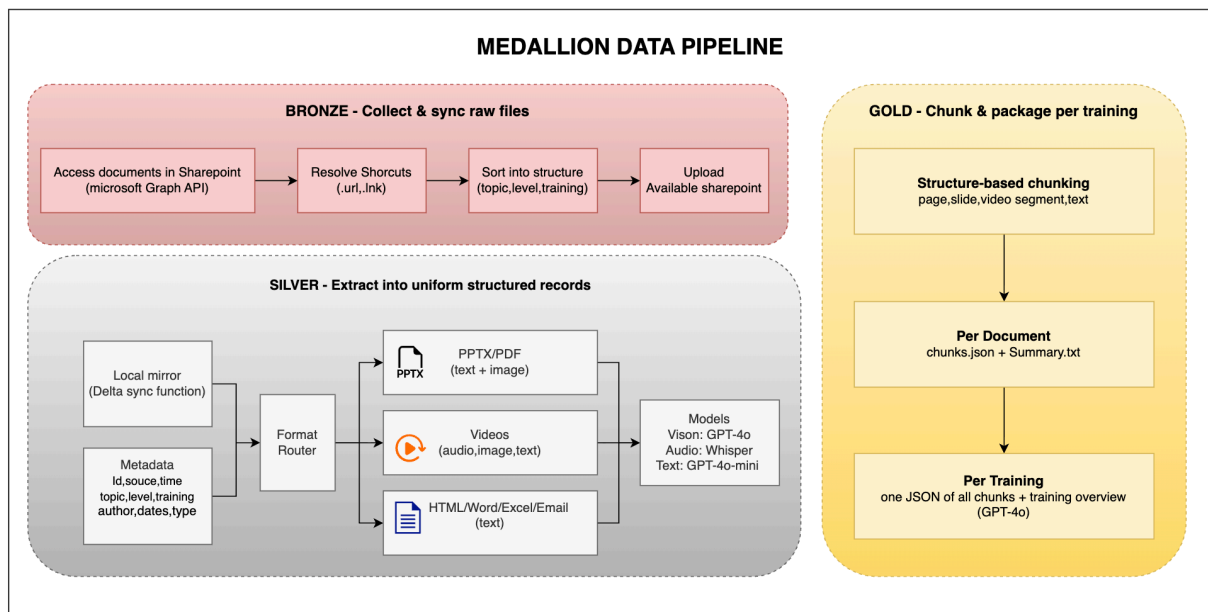


Figure 5.3: The data preparation pipeline, organised as a medallion architecture. Raw training files are collected from several SharePoint libraries into a synchronised local mirror (Bronze), extracted and standardised into uniform structured records using GPT-4o and Whisper (Silver), and assembled into per-training collections of chunks (Gold). The Gold chunks are the input to the retrieval stage, where they are embedded into a vector index and arranged into a knowledge graph.

Separating the work into these layers keeps each concern in one place and makes the pipeline easy to rebuild. Raw files are never altered, cleaning and format handling happen only in the Silver layer, and only the Gold layer produces the artifacts used downstream. Every step is incremental, so once the corpus has been built, later runs process only the files that have changed.

5.2.3 Bronze Layer

The pipeline begins with the data as it already exists inside the company. Before any of it can be summarised or indexed, the raw material has to be gathered from its original locations and brought into a consistent state. This is the work of the Bronze layer.

The training material is stored in several SharePoint libraries and comes in different formats. Most of these formats, such as PowerPoint decks, video recordings, PDFs and Word documents, are readable as they are and need no special treatment. Two formats are the exception, because they do not contain the document itself. Web shortcuts are links to Docupedia⁸ pages, and Windows shortcuts are only pointers to a file stored

⁸Docupedia is the internal enterprise wiki of Bosch, based on Confluence.

elsewhere. Both are handled during collection. Each web shortcut is exported to HTML so that the page content is captured, and each Windows shortcut is resolved by finding the original file from its path and downloading that file. Access to SharePoint for all of this is handled through the Microsoft Graph API⁹.

Once the formats are handled, the files are sorted into a fixed hierarchy of topic, then level, then training module. This order is the Bosch Rexroth standard, and it also supports the later construction of the knowledge graph. The sorted files are then uploaded to one consolidated SharePoint library, which from this point on serves as the single source of record for the corpus. Downstream processing does not read from the cloud directly. Instead, the consolidated library is mirrored to a local copy, because reprocessing files locally is faster and avoids repeated downloads of a large library on every run. The mirror is kept up to date incrementally. A small manifest records the path of each file together with its entity tag, or eTag¹⁰, which SharePoint changes whenever a file changes. On each run the manifest is compared against the library. New files are downloaded, files that were removed in the cloud are deleted locally, and files whose eTag is unchanged are skipped.

The result of the Bronze layer is a local mirror of every source file, still in its original format, together with the manifest that keeps it aligned with the cloud. The corpus is now complete and current, but it is still a set of raw files in many different formats. Converting those files into clean and uniform text is where the Silver layer takes over.

5.2.4 Silver Layer

The Bronze layer produces a complete local copy of the source files, but those files are not yet usable for retrieval. The purpose of the Silver layer is to transform the information contained in these files into a single, uniform textual form. This is the most involved part of the data preparation, because each format stores its content in a different way and needs its own method to recover it. It relies on two hosted models, GPT-4o¹¹ for reading text and images and Whisper¹² for transcribing speech, both served through Azure OpenAI¹³.

⁹Microsoft Graph is the unified REST API through which applications access Microsoft 365 data, including SharePoint document libraries. See <https://learn.microsoft.com/graph>.

¹⁰An entity tag (eTag) is an HTTP identifier that changes whenever a resource is modified, which allows efficient change detection without downloading the file.

¹¹GPT-4o is a multimodal large language model of OpenAI that accepts both text and images.

¹²Whisper is the automatic speech recognition model of OpenAI [39].

¹³Azure OpenAI Service provides access to OpenAI models such as GPT-4o and Whisper hosted inside Microsoft Azure, which keeps the data within the own cloud tenant of the organisation.

Not every format is accepted. The formats that are processed are PowerPoint presentations, PDF documents, recorded videos, exported web pages, Word documents, spreadsheets and email exports. Any other format is skipped in this layer, and the names of the skipped files are written to a separate text file rather than being dropped silently. This record lets the developers review later what was left out and decide whether a new format should be supported in future versions.

Presentations and PDF documents are handled through a common path. A presentation is first converted into a paged document, so that slides and PDF pages can be treated in the same way. Each page is then read on two levels. Its written text is taken directly from the page, and the page is also rendered as an image and passed to GPT-4o, which describes the visual content that plain text cannot capture, such as diagrams, screenshots and figures [CITE: [multimodal / visual document understanding](#)]. The pages are read in sequence, and the summary of the earlier pages is given to the model as context for the next one, so that the description of a deck stays consistent from page to page.

Recorded videos carry information in two channels at once, the spoken narration and the slides shown on screen, and both are recovered. The audio track is separated from the video and transcribed by Whisper, which also returns the time span of each spoken segment. Long recordings are split into shorter parts before transcription and joined again afterwards, with the timestamps adjusted so that they remain correct. In parallel, the video is sampled at a fixed interval to capture the slide visible at each point, and each captured slide is described by GPT-4o. The recording is then divided into segments, and for every segment three parallel views are kept, the spoken transcript, the visual description of the slide, and a consolidated account that combines the two. Every segment keeps the start and end time it covers, so that a later answer can be traced back to the exact moment in the recording.

The remaining formats contain mostly text and need less interpretation. Exported web pages, which originate from Docupedia, are first converted into a lightweight markup form¹⁴ that preserves their tables, headings and lists, so that the structure of the page is not lost. Word documents, spreadsheets and email exports are read directly, together with any attachments. In every case the recovered text is then passed to GPT-4o for the same treatment as the text taken from a slide or a video.

¹⁴Markdown is a lightweight markup language that preserves structure such as headings, lists and tables in plain text.

Across all of these formats, the model is never asked for free text. Each call is constrained to return a fixed JSON structure with a short title and a description of the content, and this is the step that turns unstructured material into structured records. The instruction also asks for the output in English, which has the useful effect of normalising the German and the English material into one language [CITE: cross-lingual retrieval]. The full prompts are given in the appendix. The model is called with deterministic settings, so that the output is as reproducible as possible.

Every record is stored together with a set of metadata, which is a small structured description that sits alongside the text of the record. Its purpose is to let the later stages work with a record without reopening the original file or having to guess where the record came from.

This metadata falls into a few kinds. The first identifies the record and its origin. It gives each record a unique identifier and states which file it was taken from, which page or which video segment it represents, and, for video, the exact start and end time it covers. This makes every record traceable, so that an answer can later point back to the precise slide or moment it came from. The second kind places the record in the training hierarchy by recording its topic, its level and its training module. The third kind is taken from SharePoint through the Microsoft Graph API and describes the document at its source, including the author, the creation and last modification dates, the original file type and the web address. This last kind is what the trust score, described in the trust scoring section, later builds on.

The metadata also records how each record connects to the records around it. Each record points up to its parent, the larger summary it belongs to, and each summary points down to its children, the smaller records beneath it. Records that follow one another also point to their previous and next neighbour, so that the reading order of a document is preserved and a slide still knows which slide came before it and which comes after. Finally, records that capture the same content in different ways, such as the transcript, the slide and the combined view of one video segment, are linked to one another as related records.

These links matter because they turn a flat collection of records into a connected structure. During retrieval the system can begin at one relevant record and then move upward to its parent for broader context, downward to its children for more detail, or sideways to its neighbours and related records for continuity and for the other views of the same content. The same links also form the backbone of the knowledge graph [13],

where the parent, child and related relationships become the edges that join documents, sections and chunks.

By the end of the Silver layer, the corpus no longer exists as slides, videos and web pages. It exists as uniform, structured records, each written in English and carrying a full description of where it came from. What these records still lack is a form suited to search, since a whole document is too large to return as a single result. Turning them into a clean collection of chunks, ready to be indexed for retrieval, is the task of the Gold layer.

5.2.5 Gold Layer

The Silver layer produced clean, uniform records, but they are still spread across the pipeline one file at a time. The aim of the Gold layer is to bring these records together into their final form, so that each training becomes a single, tidy package that the retrieval system can pick up and use. This is the last step of the data preparation, and once it is done the corpus is ready to be searched.

5.2.5.1 Chunking

The content of each document is divided into small pieces called chunks. The split follows the natural structure of the document rather than a fixed length, so each chunk is a part that already stands on its own [40]. A slide deck or a PDF is divided page by page, so that every slide or page becomes one chunk. A video is divided along time, so that each timed segment of the recording becomes one chunk. A web page or an email becomes a single chunk, since it has no page structure to split on, and it is broken down further only when it is embedded for search¹⁵. Splitting by structure in this way keeps each chunk centred on a single idea, which is more useful for retrieval than cutting the text into equal blocks that might break in the middle of a point.

Each chunk holds three things together, the summary that GPT-4o produced for that piece, the original text it was summarised from, and the metadata described in the Silver layer, such as its identifier, its source, its position and its links. Because a chunk carries its own metadata, it can be understood and traced on its own, without opening the document it came from.

Every document produces two files at this stage, a JSON file that holds its chunks and a text file that holds its summary. The chunks of all the documents that belong to the

¹⁵LangChain is an open-source framework for building applications with large language models. Its text splitter utilities divide long text into overlapping character-based chunks at embedding time, as described in the retrieval implementation section.

same training are then collected into one JSON file for that training, so a training is no longer a set of separate documents but a single ordered collection of chunks.

An overview of each training is also generated by GPT-4o. The individual document summaries are passed to GPT-4o, which condenses them into a single description of the training as a whole.

The result of the Gold layer, and of the data preparation as a whole, is a clean collection of chunks. Each chunk is written in English and carries its full metadata and its links to the chunks around it. The corpus is now ready to be made searchable. In the retrieval stage, described in the next section, these chunks are embedded into a vector index¹⁶ and arranged into a knowledge graph, the two representations over which the system retrieves.

5.3 Retrieval Implementation

The data preparation ends with a clean collection of chunks, but chunks on their own cannot answer a question. For each query the system has to find the few chunks that matter and put the best of them in front of the language model. This is the task of retrieval. Retrieval here looks at the corpus through two eyes. One eye searches by meaning over a vector index, and the other looks through the knowledge graph. Each eye finds things the other can miss, so their results are combined and then reordered before they reach the model.

This section describes how each chunk is turned into a vector and kept in a vector database. It then looks at how the two eyes search, the vector eye on its own and the graph eye on its own, and how their two views are joined into one through a fusion step. It ends with the reranker, which puts the final chunks in the best order before they reach the model.

5.3.1 Embedding

The baseline embedded each chunk with a single small, English-only dense model, BAAI/bge-small-en-v1.5, and stored the result as one dense vector. Two properties of the corpus made this a bottleneck. First, the material is bilingual, and an English-only model represents the German content poorly. Second, industrial training is full of exact identifiers such as process codes and machine numbers. A purely dense vector blurs two

¹⁶The vector index is a Qdrant store of embeddings produced by the BGE-M3 model [12].

similar looking codes and can return the wrong document for an exact code query. Both weaknesses work against the reliability the assistant is meant to improve.

The model was therefore replaced with BGE-M3¹⁷ [12]. BGE-M3 is multilingual, so it handles the German and the English material in one model. It produces a dense vector for meaning together with a sparse vector for exact words in a single pass, so codes are matched on the sparse side while meaning is matched on the dense side. It is also a larger model with a longer input limit, which gives stronger representations of the longer enterprise documents. Producing both vectors from one model, instead of adding a separate keyword index beside the dense one, keeps the pipeline simple to scale. BGE-M3 is built on a multilingual transformer encoder¹⁸, has about 570 million parameters, produces a dense vector of 1024 dimensions, and accepts up to 8192 tokens. It is downloaded from the Hugging Face Hub and run locally, so the training data never leaves the machine to be embedded, unlike the GPT-4o calls in the Silver layer. Dense vectors are compared by cosine similarity and sparse vectors by the weighted overlap of their terms. The pipeline is implemented in Python with LangChain.

Two smaller measures make the representation fairer to short documents. Before a chunk is embedded, the stored JSON is cleaned back into plain prose and a short identity line with the code, title and level of the document is placed in front of it, so the code becomes part of the embedded text. A single stronger vector is also built for each document from its code, title and description, so that short overview documents are not lost to longer ones that contribute many more chunks. Table 5.4 summarises the change from the baseline.

Table 5.4: Baseline and enhanced retrieval representation.

Aspect	Baseline	AVAILABLE 2.0
Embedding model	BAAI/bge-small-en-v1.5	BGE-M3
Language coverage	English only	Multilingual, German and English
Vectors per chunk	One dense vector	One dense and one sparse vector
Exact code matching	Blurred by the dense vector	Handled by the sparse vector
Vector store	FAISS, a file loaded into memory	Qdrant, a database service on disk

¹⁷BGE-M3 (BAAI General Embedding) is a multilingual model that produces dense, sparse and multi-vector representations in one pass.

¹⁸BGE-M3 is built on the XLM-RoBERTa architecture, a multilingual transformer encoder.

5.3.2 Vector Database

The baseline stored its vectors in FAISS, a similarity search library that indexes dense vectors only. FAISS could no longer hold the full representation, since adding the keyword signal would have meant running and synchronising a second, separate index beside it. FAISS is also just a file that the program loads into memory, and not a database that runs on its own, which makes it hard to serve a large enterprise knowledge base.

For these reasons the vector store was moved to Qdrant¹⁹, which supports hybrid search directly. Qdrant keeps the dense and the sparse vector of every chunk in one place and combines them at search time, so exact code matching and meaning based matching stay together. It runs locally in a Docker container as its own service, saves the index on disk, and scales more easily as the corpus grows. Inside Qdrant the dense vectors are kept in memory and compared by cosine similarity, while the sparse vectors are kept on disk to save memory.

Each chunk is also split into smaller pieces of about a thousand characters with a little overlap, and only these pieces are stored and searched, because a short piece is easier to match well. The full chunk is kept aside, and when one of its pieces is matched, the whole chunk is returned. The search stays precise, and the model still receives the full context.

5.3.3 Vector Eye

The first eye searches the vector database. When a question comes in, it is embedded with the same BGE-M3 model into a dense and a sparse vector, and both are sent to Qdrant. Qdrant runs two searches at once, one that matches the question by meaning through the dense vectors and one that matches it by exact words through the sparse vectors. This hybrid search is the main change from the baseline, which searched by meaning alone and often missed an exact code or machine number named in the question.

The two searches produce two ranked lists, which Qdrant merges into one with reciprocal rank fusion [35]. This method merges by the position of a chunk in each list and not by its score. The dense and sparse scores sit on different scales and therefore never have to be compared directly, and no weight has to be set by hand between meaning and exact words.

From the merged list the vector eye keeps its top ten chunks and passes them on. Ten is a moderate depth that balances recall against the noise and the lost in the middle

¹⁹Qdrant is an open source vector database. See <https://qdrant.tech>.

effect discussed in the literature review, and it is not a value tuned on the evaluation set. These chunks are only the view of one eye, and they are combined with the graph eye before the final ordering.

5.3.4 Graph Eye

The vector eye is strong at finding chunks that look like the question, but it has a blind spot. It looks at each chunk on its own and cannot connect pieces of information that sit in different documents. Take the question “In which SAP system is the outbound delivery created?”. The sales order document mentions the outbound delivery but not the system, so it answers the question only in part. The system is explained in a separate warehouse document that shares few words with the question, and the vector eye cannot reach it. The graph eye addresses this by treating the corpus as a knowledge graph, in which chunks and the things they mention are linked. In that graph the two chunks are joined through the shared item outbound delivery, so the second chunk, the one that holds the answer, can still be found. The baseline had no such component. The graph eye is a new addition of this work, and it follows the line of graph-based retrieval methods in the literature [13], [14].

5.3.4.1 Graph Construction

The graph is built in two layers. The first layer fixes the skeleton from the folder hierarchy, and the second adds meaning on top of that skeleton. Figure 5.4 shows the schema.

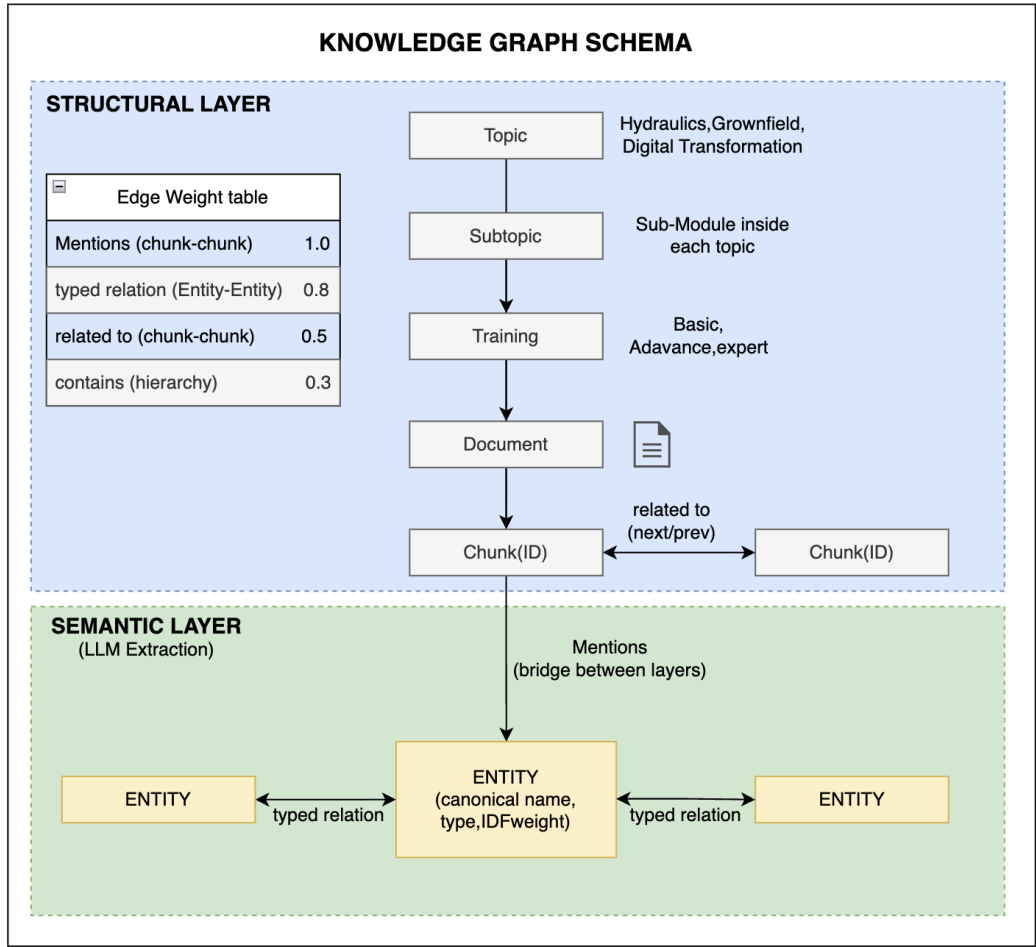


Figure 5.4: Schema of the knowledge graph with its structural layer and its semantic layer. The table lists the edge weights used during traversal.

The structural layer is built directly from the folder hierarchy and from the links already stored with each chunk. Topics, subtopics, trainings, documents and chunks become nodes, joined by `contains` and `part_of` edges, so that the graph knows that a chunk belongs to a document and a document to a training. A deterministic build was chosen over inferring this structure with a model, because the hierarchy is already recorded exactly in the data, and a model would add cost and errors with nothing to gain. This layer is cheap and exact, and it gives every later step a reliable backbone on which weight can pool inside the correct document or training.

Structure alone only links chunks that already sit close together. It cannot join two chunks in different documents that discuss the same thing, which is the very gap the graph eye is meant to close. The semantic layer adds this. The entities that the chunks talk about, and the relations between them, are extracted from the text. The chunks are grouped into small overlapping batches and passed to a large language model, GPT-4o by default. For each batch the model returns the entities it mentions and the typed

relations between them, for example requires or precedes, each tagged with a confidence and the chunk it came from. An extraction prompt fixes what is extracted and how. It requests strict JSON, and its rules keep the output grounded. Only information explicitly supported by the chunks may be used, entity labels must be short and canonical, confidences must lie between 0 and 1, and an empty result must be returned when the model is unsure. The full prompt is given in the appendix. An LLM was chosen over a rule-based or trained extractor because the vocabulary is domain specific and unlabelled, which makes a supervised tagger impractical.

The extracted entities are then merged. Entities that refer to the same thing become one node, and rarely seen ones are dropped. Each surviving entity is given a specificity weight, its inverse document frequency,

$$\text{IDF}(e) = \log\left(\frac{N}{\text{df}(e)}\right) \quad (5.1)$$

where N is the total number of chunks and $\text{df}(e)$ is the number of chunks that mention entity e [37]. An entity that appears almost everywhere therefore counts for little, while a specific one counts for a lot. Finally the merged entities are written back into the graph as entity nodes. Each is linked to every chunk that mentions it by a `mentions` edge, and to other entities by typed relation edges. Each topic ends with its own set of nodes and edges. This is what allows the two chunks in the earlier example to be joined through the shared entity outbound delivery, even though they share few words.

5.3.4.2 Graph Traversal

Retrieval on the graph works differently from vector search. Vector search scores each chunk on its own by how close its wording is to the query and returns the closest ones. It never looks at how chunks connect. Graph traversal starts instead from the concepts the query is about. The query is matched to entities in the graph, and relevance then moves along the edges to the chunks those entities link to. A chunk can be reached even when it shares few words with the query, as long as the graph joins it to a matched concept. Retrieval this way follows meaning and structure, and not surface wording.

Two engines can traverse the graph, and a mode switch picks which one runs. Both start the same way and end the same way. Each matches the query to a few seed entities in the entity index, and each returns a ranked list of chunks with scores to the fusion step. They differ only in how relevance spreads from the seeds to the chunks. Both return the same result object, so the fusion step and the reranker do not change when the engine

is switched. Figure 5.5 shows the shared seed generation and the difference between the two engines on a small example.

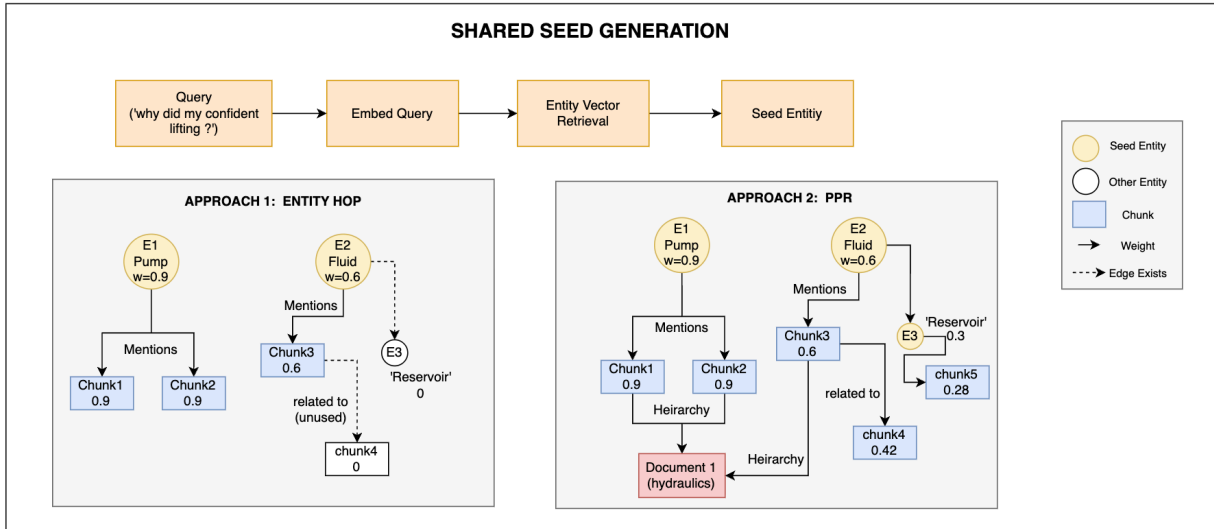


Figure 5.5: Shared seed generation for both graph engines, with the entity hop on the left and Personalized PageRank on the right. Circles are entities, squares are chunks, and the numbers are the weights that reach each node.

Seed weight is set the same way in both engines. Each seed entity is weighted by its confidence signal multiplied by its IDF specificity, so that a generic entity that appears almost everywhere counts for little [16].

5.3.4.3 Entity Hop

The entity hop matches the question to the concepts it names and then steps from those concepts to the chunks that mention them. It runs alongside the vector eye on every query and works well when the answer sits in chunks that name the concepts of the question directly.

When a question comes in, it is first compared against every entity in the graph, using a small vector index built only for entities. The closest matches become the seeds. Seeds are the concepts the question is about, and they are the only points the walk can start from, so this first match already decides how far the search can reach.

Not every seed deserves the same trust, so each one is given a weight. Two things set it. The first is the confidence of the match, taken from the semantic similarity, from a fused score that also rewards exact word matches, or from whichever of the two is higher, since either route finding the entity is reason enough to trust it. The second is how specific the entity is, measured by the IDF defined in the graph construction. The two parts are multiplied into one seed weight,

$$\text{weight}(e) = \text{signal}(e) \cdot \text{IDF}(e) \quad (5.2)$$

so that an entity that is both a strong match and rare in the corpus carries the most weight [16]. Weak matches below a floor are removed first, and the number of seeds is capped, so the walk never starts from a long tail of loosely related concepts.

Sometimes nothing in the question matches the graph well enough. When every match falls below the floor, the engine returns nothing instead of guessing. This is deliberate. The graph result is later merged with the vector result, so an empty graph answer leaves the result of the vector eye in place and does not weaken it with a poor match.

One more check happens before the walk spreads out. Each entity in the graph carries a topic label. Because some terms show up under more than one topic, a matched entity can belong to a topic that has nothing to do with the question. To handle this, the seed weights are summed per topic, and only the topic with the most weight, together with any topic close behind it, is kept. An off topic entity can then no longer pull the search into the wrong part of the corpus.

The walk now moves outward from the surviving seeds. Each seed steps across its mentions edges to every chunk that names it and passes its weight on to that chunk. A chunk named by several seeds collects weight from all of them,

$$\text{raw}(c) = \sum_{e \in S(c)} \text{weight}(e) \quad (5.3)$$

where $S(c)$ is the set of kept seeds that mention chunk c . The walk does not have to stop here. It can go further, from a chunk to its own entities and on to more chunks, but two rules keep this in check. The weight shrinks at each extra step by a decay factor d between 0 and 1, so a chunk reached at hop h contributes only $\text{weight}(e) \cdot d^{h-1}$, much less than a chunk right next to a seed. Entities linked to a very large number of chunks are also skipped in these later hops, since such a hub connects to almost everything and would spread weight without adding real signal. By default the walk stays at a single hop.

Once every chunk has its raw score, two adjustments are made. The first stops long chunks from winning just by size. A chunk that mentions many entities has more chances to pick up weight, so its score is divided by the square root of the number of entities it holds,

$$\text{norm}(c) = \frac{\text{raw}(c)}{\sqrt{|E(c)|}} \quad (5.4)$$

where $|E(c)|$ is the number of entities the chunk mentions. The second adjustment gives a chunk a small boost when the best chunk in its own document already scored highly,

$$\text{score}(c) = \text{norm}(c) \cdot (1 + \beta_d \cdot \text{prior}(d_c)) \quad (5.5)$$

$$\text{prior}(d_c) = \frac{\max_{c' \in d_c} \text{norm}(c')}{\max_d \max_{c' \in d} \text{norm}(c')} \quad (5.6)$$

where d_c is the document that holds chunk c and β_d sets how strong the boost is. The prior compares the best chunk in that document against the best chunk in the whole corpus, so it always lies between 0 and 1. A chunk is therefore judged partly on its own strength and partly on the strength of its document.

Finally, chunks that fall too far below the top score are dropped, no single document is allowed to take too many places, and only the top k chunks are kept. A smaller k is used when the seed confidence was low, so that a weak match cannot fill the result with noise.

5.3.4.4 Personalized PageRank

The entity hop can take more than one step, but it never looks at how the whole graph is connected. It adds up whatever weight lands on each chunk within a few jumps of the seeds and stops. A chunk that is only loosely reachable can still score well because a short path happens to touch it, while a chunk that is well connected to the question through many small paths can be missed.

PageRank was introduced to rank web pages by their links and not only by the words they held. A page counted as important if many pages linked to it, and more so if those pages were important themselves. Importance came from how the whole web pointed to a page and not from the page alone [17]. Plain PageRank, however, gives the same ranking for every user. Retrieval needs something narrower. It does not need to know which chunk is important in general, but which chunk is important for this question. Personalized PageRank does exactly that. It lets importance flow across the graph in the same way but keeps pulling it back toward a chosen starting point, and in this work that starting point is the set of concepts the question is about [16], [18].

When a question comes in, the engine first has to decide where the weight should start. It looks up the concepts of the question in the entity index in the same way as the entity hop, and these become the seeds. Two small checks keep the seeds clean. Entities whose match is too weak are left out, although if none are strong enough the best few are kept anyway, so that the walk always has somewhere to begin. Entities that show

up in far too many chunks are dropped, since a concept that appears everywhere says little about this one question. What remains is merged, so that different spellings of the same concept fold into one seed, and that seed keeps the weight of its strongest version instead of adding them up. Each seed carries the same signal and IDF weight as before, so a strong and specific concept starts with more weight than a weak or generic one.

The seeds do not have to come from the graph alone. The best chunks of the vector eye can be added as extra starting points, weighted by their search score, so that the walk begins partly from the strong vector signal without the two legs fully merging. By default these chunk seeds take a fifth of the starting weight and the entity seeds take the rest. All of this is gathered into one starting vector e across the nodes of the graph, scaled so that it adds up to one,

$$\sum_i e_i = 1, \quad e_i \geq 0 \quad (5.7)$$

This vector is where the walk keeps returning to. If none of the seeds land in the graph of a particular topic, that topic is skipped. Now the weight begins to flow. From the seeds it spreads out across the graph step by step, and at each step a part of it is pulled back to the seeds, so that the walk never drifts too far from what the question was about,

$$\pi = \alpha \cdot S \cdot \pi + (1 - \alpha) \cdot e \quad (5.8)$$

Here π is the weight that has settled on each node and α decides how far the weight may wander before it is pulled back. A higher α lets it travel further across the graph, a lower one keeps it near the seeds, and the value used here is 0.5. The flow is repeated until the weights barely change from one round to the next.

As the weight flows, it does not treat every connection the same, and the strength of an edge depends on its type. A `mentions` edge, which joins a chunk to a concept it talks about, is the strongest signal and carries full weight. A relation between two concepts carries a little less, and less again when the model was unsure about that relation. A link between neighbouring chunks carries less still, and the structural links of the folder hierarchy carry the least, just enough to keep weight pooling inside the right document. The strength of a single edge between nodes i and j is

$$A_{ij} = \begin{cases} 1.0 & \text{for a mentions edge} \\ 0.8 \cdot \text{conf} & \text{for a typed relation between entities} \\ 0.5 & \text{for a related to edge between chunks} \\ 0.3 & \text{for a hierarchy edge} \end{cases} \quad (5.9)$$

where conf is the confidence the model gave to the relation. The flow needs one more rule, which says how a node shares its weight with its neighbours. A node passes on all the weight it holds, split across its edges in proportion to their strength. This is captured by taking the edge weights in the matrix A and normalising each column,

$$S = AD^{-1}, \quad D_{jj} = \sum_i A_{ij} \quad (5.10)$$

so that every column adds up to one. The effect is quiet but important. A node with only a few edges hands each neighbour a healthy share, while a node connected to almost everything splits its weight so thinly that each neighbour barely feels it. A generic, over-connected node therefore cannot take over the walk, in the same way as IDF stops common entities from dominating the seeds.

When the flow settles, every node holds some amount of weight, and the chunks that collected the most are the ones best connected to the question. Each topic was walked on its own graph, so the results are then brought together according to how strongly the question seeded each topic. A topic that drew more seed weight counts for more, the weight of each chunk is scaled by the share of its topic, and a chunk that appears under more than one topic keeps its highest value,

$$\text{score}(c) = \max_t \pi_t(c) \cdot \text{share}_t, \quad \text{share}_t = \frac{\text{raw}_t}{\sum_{t'} \text{raw}_{t'}} \quad (5.11)$$

Only the chunks are read out for the final list, although the documents also collect weight along the way, and the strongest of them are shown as recommended reading.

Finally the list is trimmed in the same way as in the entity hop, so that the rest of the system sees one consistent kind of result. Chunks far below the top score are dropped, no single document is allowed to take too many places, and the top k are kept, fewer when the seeds were weak to begin with. Because the weight moves through the whole graph at once instead of in fixed jumps, this engine can settle on a chunk several relations away from any seed, the case that the entity hop keeps missing. In the example of Figure 5.5 the entity hop gives no weight to chunk 4 and chunk 5, while Personalized PageRank reaches both through the related to edge and through the second entity.

5.3.5 Hybrid Retrieval

Hybrid retrieval combines the vector eye and the graph eye. The vector eye is strong on semantic meaning, and the graph eye is strong on connections through shared concepts. The hybrid step brings their two views together. It merges their two ranked lists into

one pool, orders that pool by how well each chunk answers the question, and passes the best chunks on to the generation step. Figure 5.6 shows the flow.

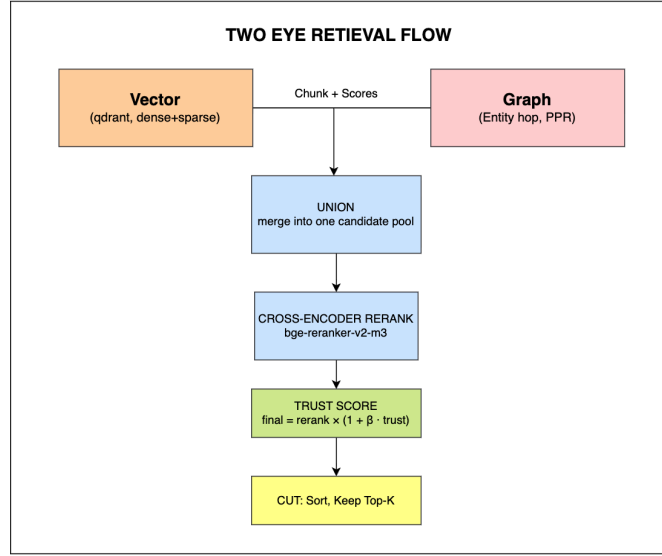


Figure 5.6: The two eye retrieval flow from the two legs to the final top-k chunks.

5.3.5.1 Fusion

Both legs run, and their chunk lists are merged into one candidate pool by a union. The chunks are taken one from the vector eye, then one from the graph eye, and a chunk found by both legs appears only once. A union is used because neither leg can bury the other this way, which is the risk when one list is longer or trusted more than the other. Writing the vector list as $V = (v_1, v_2, \dots)$ and the graph list as $G = (g_1, g_2, \dots)$, the pool is

$$\text{pool} = \text{dedup}(v_1, g_1, v_2, g_2, v_3, g_3, \dots) \quad (5.12)$$

where dedup keeps the first appearance of each chunk. No score is attached yet. This order only decides which chunks make the shortlist, and the reranker sets the real order next.

5.3.5.2 Reranking

The shortlisted chunks are re-scored by a cross-encoder model [CITE: BAAI/bge-reranker-v2-m3]. The first stage of retrieval embeds the question and each passage separately, which is fast but only rough. The cross-encoder reads the question and one chunk together and judges how well that chunk answers the question [38]. This is much more precise, and it is affordable because it runs only over the small pool and not over the whole corpus.

The last part folds in trust and cuts the list. Each reranked score is adjusted by the trust of the document a passage comes from, as described in the trust scoring section, so that chunks from more trustworthy documents move up,

$$\text{final} = \text{rerank} \cdot (1 + \beta_t \cdot \text{trust}) \quad (5.13)$$

where β_t sets how strongly trust influences the order. The pool is then sorted, the top chunks are kept and sent to the generation step, and the unique documents behind them are shown to the user as recommended reading. The two eye combination and the cross-encoder are both new to this work. The baseline used a single vector leg, with no fusion and no reranking.

5.4 Trust Scoring Implementation

Retrieval and reranking bring back the passages that are most relevant to a question, but relevance is not the same as reliability. Two documents can both match a question while differing in how far they can be trusted. One might be a current internal handbook, the other an outdated draft written by someone unknown. Both look equally good to a search that only cares about wording and concepts, so something more is needed to tell them apart.

This matters because of how the knowledge base is used. It acts as a single shared source that many internal projects draw from, and it keeps growing over time. As it grows, older material goes out of date, training documents from unknown authors find their way in, and unnecessary files pile up next to the good ones. Nothing in the baseline measures how reliable a document is, or how well it fits with the rest of the material around it. The trust score is added to fill this gap. It does not check whether the facts inside a document are right or wrong. It estimates how reliable a document is from its content quality, its metadata, and how well it lines up with the rest of the knowledge base. That estimate is then folded into the reranking step, so that more reliable sources are favoured.

The score is built from two parts. The first is a prior, which measures the quality of a document on its own terms. The second is a support score, which measures how strongly other trusted documents back the document up. The two are blended, with the prior kept in charge,

$$\text{TrustScore} = 0.7 \cdot \text{prior} + 0.3 \cdot \text{support} \quad (5.14)$$

so a document earns most of its score from its own quality and is then nudged up or down by how well the rest of the corpus supports it. Figure 5.7 shows the full computation.

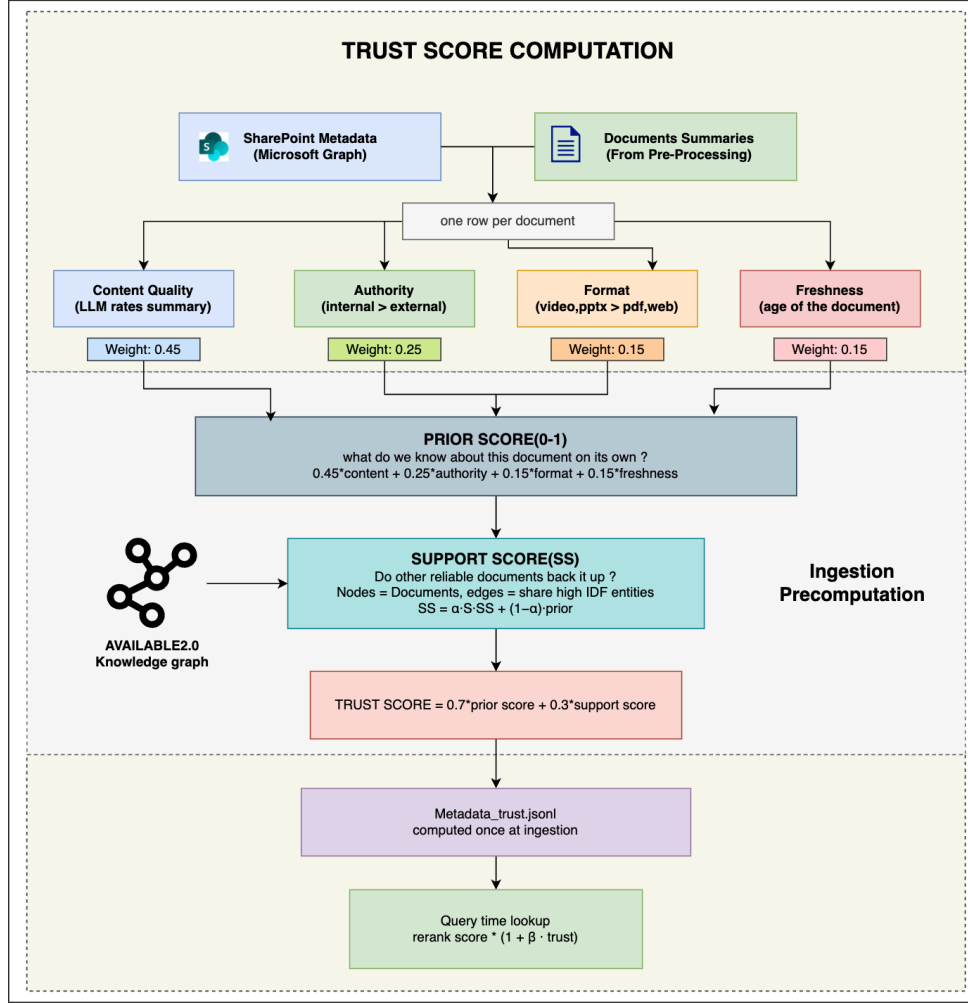


Figure 5.7: Computation of the trust score. The prior is built from four pillars, the support score spreads trust across a document graph, and the final score is stored once at ingestion and read back at query time.

5.4.1 The Prior

The prior measures what is known about a document on its own. It combines four pillars, each a value between 0 and 1,

$$\text{prior} = 0.45 \cdot \text{content} + 0.25 \cdot \text{authority} + 0.15 \cdot \text{format} + 0.15 \cdot \text{freshness} \quad (5.15)$$

and the weights add up to one, so the prior also lies between 0 and 1.

The content pillar carries the most weight, because it reflects how useful the document actually is as training material. It is scored by a language model that reads the generated summary of the document and not its raw text, since the summary is the cleaned form that the pipeline has already produced. The model rates four things, the technical depth,

how complete the procedure is, how usable the steps are, and how focused the document stays. These ratings are averaged into one content score. The full prompt used for this rating is given in the appendix. Videos receive the full content score by policy. This follows a decision of the company, since the videos are produced by their own experts, so their content quality can be assumed to be high even though a text summary does not capture their real worth.

The other three pillars come from metadata. The authority pillar rewards a known source. A document created or edited by an internal account scores highest, one from an external vendor scores lower, and one from an unknown author scores lowest. The format pillar prefers structured, text based documents over image only slides, simply because the pipeline can read text but cannot read a picture of a slide. The freshness pillar gives a small reward to recent edits. It decays slowly, with a half-life of three years, down to a fixed floor, since an old document is not automatically a wrong one,

$$\text{freshness} = \max(f_{\min}, 2^{-\text{age}/3}) \quad (5.16)$$

where the age is measured in years and $f_{\min}$ is the floor. A document with no date receives a middle value.

5.4.2 The Support Score

The quality of a document on its own is not the whole picture. A claim that many trusted documents also make is more believable than one that stands completely alone, so the second part measures this backing [23], [24]. To do this, a second graph is built, this time with one node per document. Two documents are joined when they mention the same entities, and each shared entity adds its inverse document frequency to the link, so that sharing a specific concept counts far more than sharing a common word,

$$A_{ij} = \sum_{e \in E_i \cap E_j} \text{IDF}(e) \quad (5.17)$$

where E_i is the set of entities mentioned in document i . Entities that are found in more than half of the documents are dropped as too generic, and an entity that is found in only one document is ignored, since it cannot connect anything.

Trust then flows across this document graph with the same Personalized PageRank method as in the retrieval leg, but for a different purpose [18]. In retrieval the walk started from the entities of the query and ranked chunks. Here it starts from the metadata prior and ranks whole documents. The prior seeds the walk, and trust spreads from reliable documents to the ones that back them up,

$$t = \alpha \cdot S \cdot t + (1 - \alpha) \cdot p \quad (5.18)$$

where t is the trust on each document, S is the document to document transition matrix built from the shared entity links in the same way as before, p is the metadata prior used as the seed, and α is the damping factor, set to 0.5. The result is scaled so that the most supported document scores one,

$$\text{support}(d) = \frac{t_d}{\max_j t_j} \quad (5.19)$$

A document with no entities, such as a video without text, has nothing that connects it to others, so it simply keeps its prior.

5.4.3 Limits of the Method and the Choice of Weights

This method is useful, but it rests on assumptions that do not always hold, and three limits are worth stating plainly.

The first is that it rewards the majority, and agreement is not the same as truth. Trust flows toward documents that overlap with many others. If several documents repeat the same outdated or wrong fact, the walk lifts all of them together, while a correct document that stands alone with few neighbours is left near its prior. For a corpus that is made partly of drafts and vendor slides, assuming that the crowd is right is not safe.

The second is that the edges only measure co-mention and not agreement. An edge is built whenever two documents share entities, but two documents that both mention the outbound delivery might say opposite things about it. The walk cannot tell backing from disagreement, so a document that repeats a claim and one that disputes it are treated the same, as long as they share the entity. The graph therefore only shows that documents talk about the same specifics, which is a rough stand-in for actually supporting each other.

The third is that the seed is the prior itself, so the support score is only partly independent of the quality that it is meant to add to.

These limits are the reason why the prior is kept in charge. The support score is given a weight of only 0.3 in the final blend, while the prior holds 0.7, so that corroboration can nudge a score but never override the measured quality of a document. A document that never entered the graph keeps its prior.

5.4.4 Use at Query Time

The whole trust score is computed once, when a document is ingested, and it is stored in the metadata of the document. At query time it is read back during the reranking step, where the score of each passage is multiplied by $(1 + \beta_t \cdot \text{trust})$, so that passages from more trusted documents rise in the final order. How the trust score is tested, and whether it behaves as intended, is covered in the evaluation chapter.

5.5 Evaluation

The previous sections set out how the system works, from retrieving chunks and writing an answer to scoring how much each document can be trusted. Those sections explain what was built and why each choice was made. This section describes how the performance of the system is measured. Instead of reading a few answers and judging them by feel, a fixed set of questions is run through the real pipeline and the results are scored, so that different retrieval approaches and the two versions of the system can be compared on equal terms.

The evaluation is split into two parts. The first part measures retrieval quality and asks whether the system finds the right document for a question. The second part measures generation quality and asks whether the written answer is good. The trust score is handled separately. It is judged neither on retrieval nor on answer quality, but on whether it is accurate and whether it improves the system, so it has its own evaluation. The results of all experiments are reported in the experiments and results chapter.

5.5.1 Evaluation Datasets

The questions used for evaluation come from two sources. The first is written by internal experts, and the second is a synthetic set built for this work. The training material covers three topics, which are Grownfield, the SAP S/4HANA training, Digital Transformation and Hydraulics. To test the system against real needs, the teams behind these topics were asked for questions that their own users would ask, each paired with the correct answer and the document it should come from. The Digital Transformation team was not available during the period of this thesis, so expert questions could be collected only for the other two topics. The Grownfield team contributed 40 questions across the SAP modules, a mix of single document and multi document cases, and the Hydraulics team contributed 59 questions drawn from its training handbook. Every question carries a ground truth answer and a golden source document, which is what allows both retrieval and generation to be scored against a known target.

These expert sets are trustworthy, but they are small. Ninety nine questions in total, with none at all for Digital Transformation, are too few to tell reliably whether the system works well or badly, and too few to break the results down by topic, question type or retrieval approach. A larger and more balanced set was needed before any real conclusion could be drawn.

Since the training material itself was available, a synthetic question set was built from it, covering all three topics including the one without expert questions. The set is built one document at a time. For each source document, a language model reads the cleaned content and writes five self contained questions, one of each of five types, factual, conceptual, procedural, application and summary, so that a single document produces a spread of difficulty and not five near copies. The full prompt is given in the appendix. Each question is paired with a ground truth answer and a few keywords, and the document it was generated from is recorded as its golden source. Every generated question is checked for these required fields before it is kept. This produced 1215 questions across the three topics, more than ten times the size of the expert sets, spread evenly across the five question types. Table 5.5 summarises both sources.

Table 5.5: Overview of the two evaluation datasets.

Source	Topic	Questions	Ground truth
Expert	Grownfield	40	Written by experts
Expert	Hydraulics	59	Written by experts
Expert	Digital Transformation	0	Not available
Synthetic	All three topics	1215	Generated by a language model

The two kinds of dataset play different roles. The expert questions give a small, human authored check that is tied to real information needs, while the synthetic questions give the scale and coverage needed to compare approaches with confidence. Because the synthetic answers are themselves written by a language model, they are treated as a broad measure and not as a perfect reference, and the expert sets remain the stricter test. One more precaution is taken with the synthetic set. The model that generates the questions and answers is not the same model that judges the answers later, since a language model tends to rate its own output more favourably than a neutral judge would. This effect is known as self preference bias, where an evaluator scores text it

produced itself higher than equivalent text from another source [41]. Using different models for generation and for judging keeps this bias out of the results.

5.5.2 Retriever Evaluation

Retrieval is checked by whether the golden source document is found. For each question the system returns a ranked list of sources, and this list is compared against the golden source that the question is known to come from. Two things matter here. The first is whether the golden source is found at all, and the second is how high it sits in the list. A correct source near the top is more useful than the same source buried far down, so its position is measured and not only its presence. Two metrics are used, Hit@k and mean reciprocal rank, both as defined in the theory chapter. Hit@k is read at a few cutoffs. Hit@1 is the strict case where the golden source is ranked first, and Hit@10 is the looser case where it lands somewhere in the top ten. Together the two metrics show how often the right source is found and how well it is ranked.

The evaluation runs in three stages. The first stage chooses the graph engine. The graph leg can be walked in more than one way, so the best way is settled before the graph is compared with anything else. Three engines are tested on the same seeds, so that only the traversal differs and the comparison stays fair. The entity hop takes a single step from the concepts of the query to the chunks that mention them. The multi entity hop repeats that step over several hops and reaches further across the graph. Personalized PageRank lets weight flow through the whole graph at once. Each engine is scored on the same questions. Because the graph leg feeds a reranker, which can reorder candidates but cannot recover a document that was never retrieved, the engine that reaches the most golden documents is fixed as the graph engine for the rest of the evaluation, and ranking quality at the top is a secondary criterion.

The second stage compares the retrieval legs. With the graph engine fixed, the three legs are compared under the same conditions. The vector leg retrieves by wording, the graph leg by connected concepts, and the hybrid leg fuses the two. This stage shows whether adding the graph to the vector leg helps the system find the right source, and whether fusing the two beats either one alone. The leg that retrieves best is taken as the retrieval setup of AVAILABLE 2.0.

The third stage compares the two versions of the system on the same questions. AVAILABLE 1.0 retrieves with vector search alone, while AVAILABLE 2.0 uses the retrieval setup chosen in the previous stage together with fusion and reranking. This final test shows what the changes of the second version achieve for retrieval as a whole.

5.5.3 Generator Evaluation

Retrieval shows whether the right source was found. Generation asks the next question, which is whether the answer written from that source is good. This cannot be judged by matching a golden document, since the same fact can be written correctly in many different ways. For this reason the answers are scored by a language model acting as a judge, through the RAGAS framework [9]. The three legs from before, vector, graph and hybrid, are each run all the way to a written answer, and those answers are scored, so that the comparison started in retrieval carries through to the final text.

Four measures are used, all defined in the theory chapter. Two of them judge the answer itself. Faithfulness checks whether the answer stays within the retrieved context or invents details, which makes it the hallucination check. Answer relevancy checks whether the answer addresses the question that was asked. The other two judge the context that was retrieved to write the answer. Context precision rewards relevant passages for appearing near the top of the list, and so captures the signal to noise ratio of what was fed into generation. Context recall checks whether the retrieved context held what was needed to answer.

With these four measures, each leg is taken to a full answer and scored on the same questions. This shows whether a leg that retrieves better also produces an answer that is more faithful, more on topic and better supported. Generation is also one half of the final comparison between the two system versions, next to retrieval.

Two limits are worth stating. The judge is itself a language model, so its scores are a strong signal and not a fixed truth. On the synthetic set the ground truth answers are generated and not written by experts, so context recall against them is a softer reference, and the expert sets remain the firmer check.

5.5.4 Trust Score Evaluation

The retrieval and generation evaluations test whether the system finds the right source and writes a good answer. Neither of them tests whether the trust score works correctly or whether it is helpful. The trust score therefore needs its own evaluation, with its own dataset, built to answer two questions. Does the score reflect the quality of a document, and does using it improve the answers the RAG gives.

Both experiments run on the Grownfield corpus, the Bosch Rexroth SAP training documentation. Grownfield was chosen for three reasons. It carries the rich metadata that the prior needs, which is an author, a date and a format. It covers many SAP process

modules, so it holds a wide spread of content. And every document in it was already approved by the Bosch Rexroth team, so the originals can be treated as trustworthy, which both experiments rely on.

5.5.4.1 Experiment 1, Detecting a Loss of Quality

The first experiment checks whether the trust score is valid. The idea is simple. If a document is made worse and its score drops, the score is measuring quality and not noise. Every real Grownfield document is already good, so poor ones had to be made on purpose. For this, 40 high quality documents were taken, and each was copied and damaged in five separate ways. Every copy carries a single kind of damage that is aimed at a single pillar, as listed in Table 5.6.

Table 5.6: The five kinds of damage used in the first experiment.

Damage	Pillar	What it does
Procedural	Content	Removes most of the steps of a procedure, so the process is left incomplete
Technical	Content	Strips out SAP specifics such as transaction codes and table names, so the technical depth is lost
Filler	Content	Injects off topic text, so the document loses its focus
Authority	Authority	Changes the author to an unknown one
Freshness	Freshness	Backdates the document by four years

The three content attacks were written by a language model, and the prompt is given in the appendix. The two metadata attacks change the author and the date directly. This produced 240 variants in all, 40 originals and 200 damaged copies, and each was scored again with the same scorer that the pipeline uses.

Only the prior was measured in this experiment, the part built from content and metadata. The support score depends on how the rest of the corpus corroborates a document, so recomputing it for a single damaged copy would mean rebuilding the whole document graph each time. Its behaviour is left to the second experiment. The score passes this test on three counts. The damaged pillar should fall. The overall prior should fall by a smaller amount, since one weak pillar is diluted by the others. And among the content sub scores, the attacked one should fall the most, which shows that the damage is felt where it was done and not as a vague drop across everything. Whether these drops are real is confirmed with a Wilcoxon signed rank test, reported with an effect size and a correction for testing several pillars at once.

5.5.4.2 Experiment 2, Helping the RAG

The second experiment checks whether the trust score is useful, meaning that it helps the RAG pick the trustworthy source when two sources disagree. A valid score is only worth adding if it changes the answers for the better, and this experiment is built to show that directly.

For this experiment, 20 fact rich documents were taken, and a poisoned twin of each was made by a language model that flips about ten of its facts and weakens its metadata. The poisoning prompt is given in the appendix. This gives a corpus of 40 documents, 20 real ones and 20 twins, where a real document and its twin answer the same question in conflicting ways. Building the conflict this way is deliberate. A twin shares the topic and the wording of the original but carries wrong facts, so any question drawn from it has one clearly correct source and one clearly wrong one, which is exactly the situation that the trust score is meant to resolve. Every flipped fact became a question with a known correct value and the wrong value of the twin, which gives 190 questions.

All 40 documents were indexed in the vector store, and each twin was placed into the real Grownfield graph, so that its support score could be computed by the same walk that is used in retrieval. Trust then enters the RAG in two places. It reweights the ranking after reranking by the factor $(1 + \beta_t \cdot \text{trust})$, as described in the retrieval implementation, and each retrieved chunk is also tagged with its trust value in the prompt, so that the model knows which source to follow when two conflict.

To isolate what trust contributes, the same questions are run three times in an ablation, once with trust off, once with the prior alone, and once with the full score of prior and support together. Comparing the three shows whether trust helps at all and whether the support score adds anything beyond the prior. Each run is checked in two ways. The retrieval check asks whether the good document is ranked above its poisoned twin. The generation check asks whether the written answer follows the correct value and not the wrong value of the twin. Because the three runs share the same questions, their differences are tested with a McNemar test, the paired test for two settings scored on the same items.

The answers are graded in three ways, by exact keyword match, by a custom language model judge whose instructions are given in the appendix, and by RAGAS. Each grader is first checked against the known correct value. This decides which grader can be trusted for the final reading, and avoids assuming that any single grader is reliable on its own.

The two experiments test both sides of the claim. The first shows that the score reflects real quality, and the second shows that it improves the answers.

5.6 Observability

A RAG pipeline handles both retrieval and generation. It fetches the relevant documents and then answers the question. Along the way it searches the vector store, walks the graph, fuses and reranks the candidates, folds in trust, and finally calls the language model. When an answer comes out wrong, the cause can lie in any one of these steps, and reading the final answer alone does not reveal which one failed. The pipeline therefore has to be monitored, so that each request can be opened up and inspected after it runs. Scattering manual log statements through the code is one way to do this, but it is slow to write and painful to debug when something breaks. A dedicated tracing tool was used instead, which helps not only with monitoring but also with later improvements.

Several tools offer this kind of tracing, and Opik was chosen because it is open source and can be run on the own infrastructure of the project. This keeps the traces and their data inside the same environment as the rest of the system and does not send them to an outside service, which matters for enterprise material that should not leave the local setup. The application talks to Opik as a client and sends everything it records to a local server, where it is viewed in a dashboard. A small set of environment variables points the client at the server, names the project under which all traces are grouped, and sets the workspace.

The three levels of recording introduced in the theory chapter map directly onto the way the system is used. A span is one function call of the pipeline, a trace is one question from entry to answer, and a thread is one session. A person usually asks several questions in one sitting, and each question runs the whole pipeline once, so every question becomes one trace and the traces of one conversation are grouped into one thread.

The recording is done with a decorator. Placing it above a function tells Opik to capture that function as a span each time it runs, taking its inputs before it executes and its result and timing after. When one decorated function calls another, the inner span is nested inside the outer one, so the shape of the pipeline is rebuilt as a tree without any manual bookkeeping. The top of the tree is the whole request, recorded as `rag_pipeline`, and beneath it sit the retrieval and generation steps. Retrieval opens its own nested spans for the vector search, the graph traversal, the fusion and its union, the reranker

and the recommended documents, so that the path of a question through the two legs is visible step by step.

Where the automatic capture is not enough, two calls add more detail. One attaches step level detail to the current span, such as how many chunks each leg returned, together with the token usage and the estimated cost of a model call. The other attaches request level detail to the current trace, such as the final answer and the session identifier. The session identifier is a single value that is created once when a session opens and reused for every question in it, which is what files the separate traces under one conversation.

Observability was not part of the baseline, which offered no way to trace or measure the pipeline. Adding it makes both the behaviour and the cost of each request transparent. This visibility is what allowed the retrieval approaches and the trust score to be understood and debugged while they were being built.

5.6.1 Guidelines for Trustworthy Documents

The trust score is only useful if the teams that write training material know what it rewards. For this reason the score is translated into a short set of writing guidelines. They follow directly from the way the score is built. The score does not check whether the information in a document is factually right. It estimates how reliable a document is from its content quality, its metadata and how well it aligns with the rest of the knowledge base. A document that follows the guidelines therefore has a higher chance of earning a good score.

Because the prior carries a weight of 0.7 and the support score a weight of 0.3, the share of each part in the final score follows from the weights of the pillars. The content pillar contributes $0.7 \cdot 0.45 = 31.5\%$, the three metadata pillars contribute $0.7 \cdot (0.25 + 0.15 + 0.15) = 38.5\%$, and the support score contributes 30%. Table 5.7 lists the three parts and what each of them rewards.

Table 5.7: Writing guidelines derived from the three parts of the trust score and their share of the final score.

Part	Share	What it rewards
Content	31.5%	Specific details such as transaction codes, table names, program names, fields and item types. A complete procedure with prerequisites, steps and outcome. Actionable steps that say what to enter or click and why. A focused document with one task or topic, free of filler and of descriptions of slide visuals.
Metadata	38.5%	A known and authorised author and no blank or unknown author. A recent last modified date, kept current by reviewing and saving the document regularly. A structured, text based format, preferred over image only slides, so that the content is machine readable.
Support	30%	Standard and consistent terminology, so that the document links into the shared knowledge graph. Facts that agree with other trusted documents. A document that shares its key facts with the wider corpus and does not stand in isolation.

The guidelines for the support score deserve a note. Using the same codes and names as other official documents is what connects a document to the others through shared entities, and without such connections it keeps only its prior. Agreement with trusted documents raises the score, but as discussed above, agreement is only a proxy for correctness. The guidelines therefore ask authors to stay consistent with authoritative sources, and they do not ask them to copy the majority.

6 Experiments and Results

This chapter presents the results of the evaluation designed in the methodology. The results are reported in the order in which the methodology introduced the components. Retrieval comes first. It covers the choice of the graph traversal engine, the comparison of vector, graph and hybrid retrieval, and the change from AVAILABLE 1.0 to AVAILABLE 2.0. Generation follows, judged with reference-based and reference-free measures, namely faithfulness, context precision, answer relevancy and context recall. The trust score closes the chapter through two experiments, one that checks whether the score detects deliberate quality loss, and one that checks whether it helps the system resist poisoned material.

6.1 Experimental Datasets

The evaluation uses three datasets that differ in origin, language and difficulty, as described in the methodology. The Grownfield set holds 33 questions written by the expert team on the SAP training, 17 of them single-hop and 16 multi-hop, which makes it the test for reasoning across several documents. The Hydraulics set holds 59 expert questions in German, all drawn from one training handbook, which makes it a narrow and well covered topic. The synthetic set holds 1215 questions built across all three subject areas, five per source document and one of each of the five question types, and it gives the statistical weight that the small expert sets cannot.

6.2 Retrieval Results

Retrieval decides what the generator can use, and a weak retrieval stage limits every answer that follows. Two things matter here. The golden document should rank high, and it should stay inside the top results that the generator reads. Position and coverage are both measured, using Hit@ k and mean reciprocal rank as defined in the theory chapter. Hit@ k checks whether a golden document appears in the top k , and mean reciprocal rank rewards a golden document that is placed near the top and not only inside the window.

Three comparisons follow. The first holds the entity seeds fixed and changes only the graph traversal engine, so that the effect of traversal depth and Personalized PageRank can be seen on its own. The second sets vector, graph and hybrid retrieval against each

other. The third measures the change from AVAILABLE 1.0 to AVAILABLE 2.0 on the same task.

6.2.1 Graph Engine Selection

The graph traversal engine was chosen on a fixed set of Grownfield questions. Every candidate received the same entity seeds, so only the traversal step changed. Three strategies were compared, a single entity hop, an entity traversal over three hops, and Personalized PageRank over the same seeds. Figure 6.1 reports the outcome.

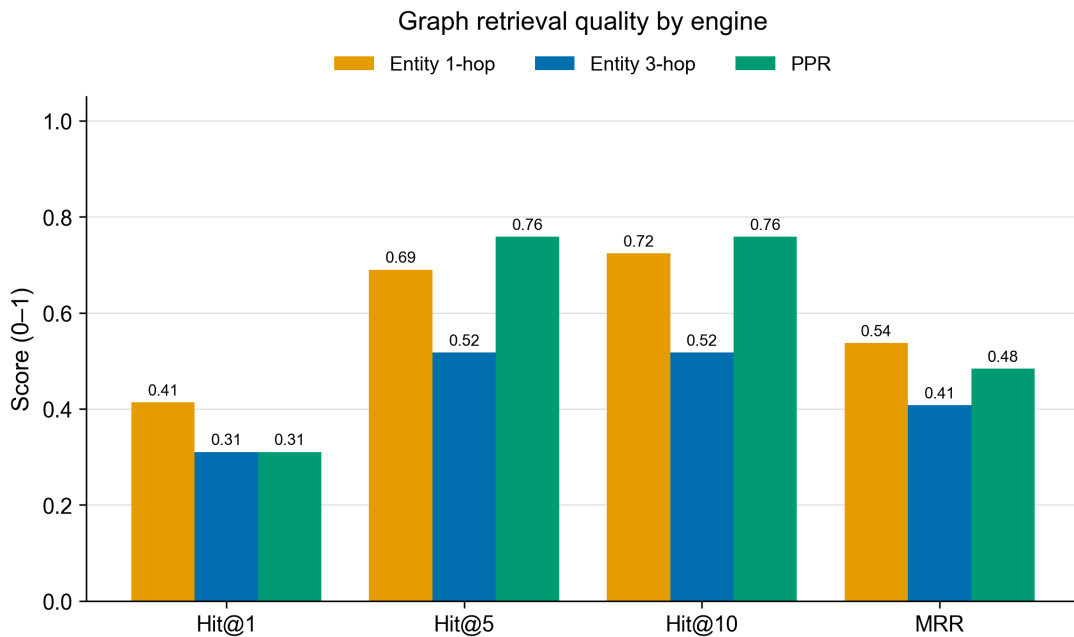


Figure 6.1: Graph retrieval quality by traversal engine, with identical entity seeds for all three engines.

The single hop was the most precise at the top. It placed the correct document at rank one most often, with a Hit@1 of 0.41, and it reached the highest MRR of 0.54. The three hop strategy performed worse on every measure, with a Hit@10 of 0.52 against 0.72 for the single hop, so stepping further out found fewer correct documents and mostly pulled in noise. Personalized PageRank spread weight across the whole graph instead of taking fixed steps. It was weaker at rank one, with a Hit@1 of 0.31, but it recovered the most correct documents overall. Its Hit@5 and Hit@10 were both 0.76, which is higher than the 0.69 and 0.72 of the single hop.

The choice came down to two strengths. The single hop is precise at the very top, while Personalized PageRank reaches wider. The graph leg does not produce the final answer. Its results pass to a reranker that re-scores every candidate at the end. A reranker can reorder the documents it receives, but it cannot recover a correct document that was

never retrieved. A wider reach therefore matters more than a clean top rank at this stage. For that reason Personalized PageRank was carried forward into the combined system. Because the reranker fixes the ordering later, the task of the graph leg is to catch as many correct documents as possible, and Personalized PageRank catches more.

6.2.2 Vector, Graph and Hybrid Retrieval

Three retrieval settings are compared. Vector retrieval ranks by dense similarity. Graph retrieval ranks by Personalized PageRank over the entity graph. Hybrid retrieval joins both candidate sets and passes the pool on to the reranker. Figure 6.2 reports Hit@10 and mean reciprocal rank for the three settings.

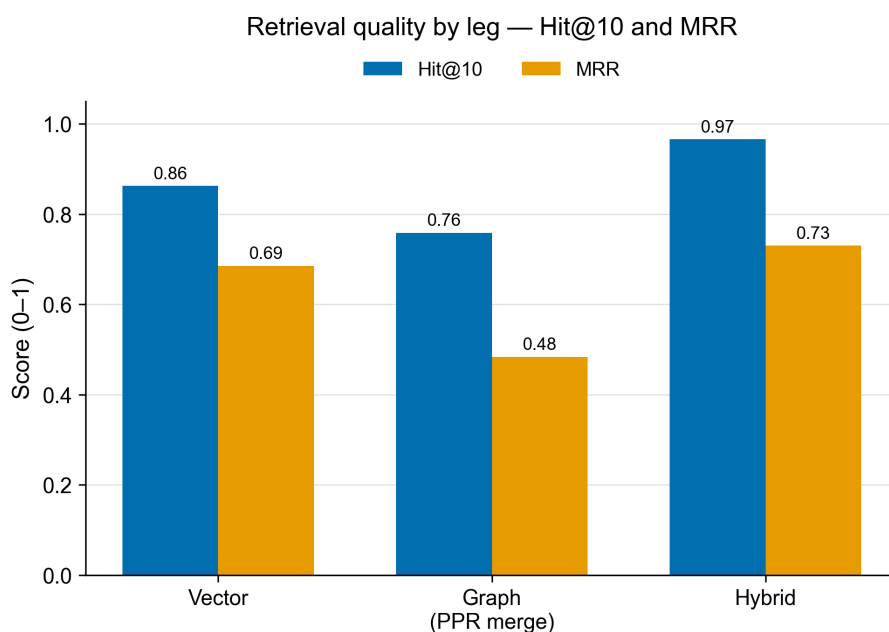


Figure 6.2: Retrieval quality of the vector, graph and hybrid legs, measured by Hit@10 and mean reciprocal rank.

Vector retrieval does most of the work. It reaches a Hit@10 of 0.86 and an MRR of 0.69, because the question and the document often use the same words, so similarity search finds the right chunks on its own. Graph retrieval alone is the weakest of the three, with a Hit@10 of 0.76 and an MRR of 0.48, and the gap in MRR is large. A likely reason is that Personalized PageRank can only start from the entities named in the question, so a question without clear entities gives it almost nothing to work with. A retriever that depends on named entities cannot stand alone when many questions are worded loosely [19]. The standalone score is therefore not a judgement on the graph as such. It shows why the graph alone is not sufficient for retrieval.

At the same time, the graph helps the hybrid setting to reach a much wider window than vector alone. Hit@10 rises from 0.86 to 0.97, an increase of eleven percentage points, while MRR rises only from 0.69 to 0.73. What the graph adds is therefore mainly recall and only slightly a better ordering. For a stage that feeds a reranker this is the right trade. The reranker can reorder anything that is already in the pool, but it can do nothing for a chunk that the pool never held. Letting more correct chunks into the pool is the more useful outcome.

Overall, the picture is steady. Vector retrieval sets the top rank and does most of the work. The graph leg adds recall that vector search misses. Hybrid keeps that gain and also ranks slightly better than vector alone. For a pipeline that passes its pool to a reranker, this is the retriever worth keeping. It widens what the reranker can choose from and leaves the parts that already work alone.

6.3 Generation Results

Generation is scored with the four RAGAS measures introduced in the methodology. Figure 6.3 reports them for the hybrid leg on Grownfield.

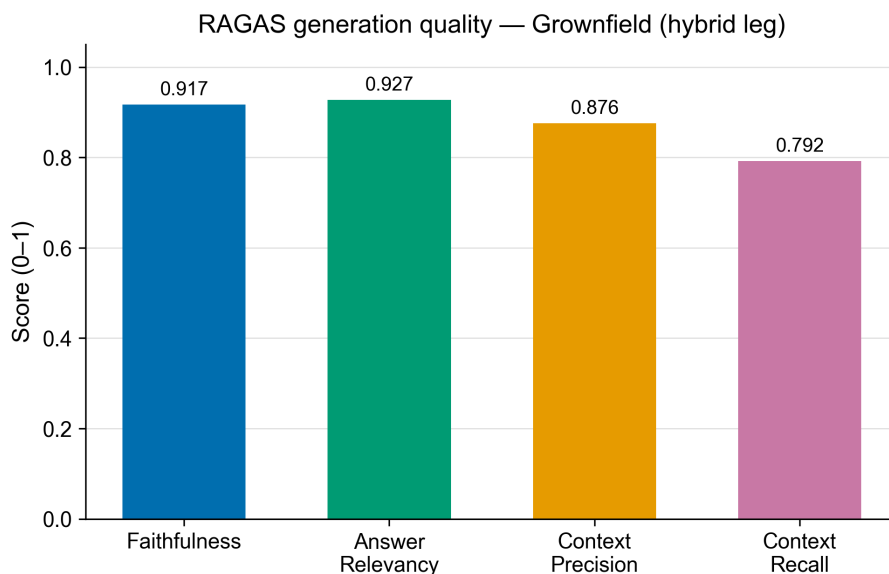


Figure 6.3: RAGAS generation quality of the hybrid leg on Grownfield.

The two measures that judge the answer are high. Faithfulness is 0.917, so about nine in ten claims of the generated answers are supported by the retrieved context, and answer relevancy is 0.927, so the answers stay on the question that was asked. The two measures that judge the context are lower. Context precision is 0.876, which means that most retrieved passages are relevant and that they appear near the top. Context recall is the lowest at 0.792, which means that in about one fifth of the claims of the reference

answers, the retrieved context did not hold the information that was needed. This points to retrieval as the main remaining limit and not to the generator.

6.3.1 AVAILABLE 1.0 versus AVAILABLE 2.0

The last comparison sets the baseline against the final system. AVAILABLE 1.0 ran a single vector retrieval pipeline, so it is compared with the hybrid setting of AVAILABLE 2.0, including fusion and reranking. The comparison is shown for each dataset in turn.

On Grownfield, the multi document reasoning set, the gain is large at every cutoff (Figure 6.4). Hit@1 rises from 0.39 to 0.66, so the correct document now leads the results far more often. The mean reciprocal rank rises from 0.46 to 0.73, and Hit@10 rises from 0.74 to 0.93. The first version left about a quarter of the questions without a correct document in the top ten, and the second version reduces this share to 7 percent.

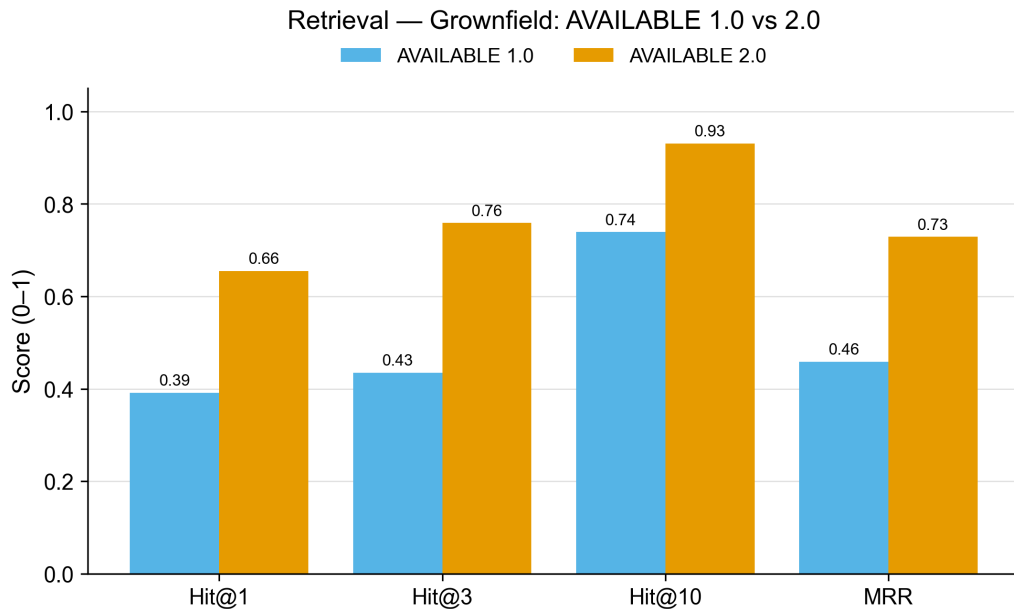


Figure 6.4: Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Grownfield.

On Hydraulics the second version is close to perfect (Figure 6.5). Hit@1 rises from 0.68 to 0.95, and Hit@3 and Hit@10 both reach 1.00, so the golden document is always among the top three results. The mean reciprocal rank rises from 0.73 to 0.97. The baseline already found the right document for most questions, but it often ranked it low, and most of the gain here comes from ranking.

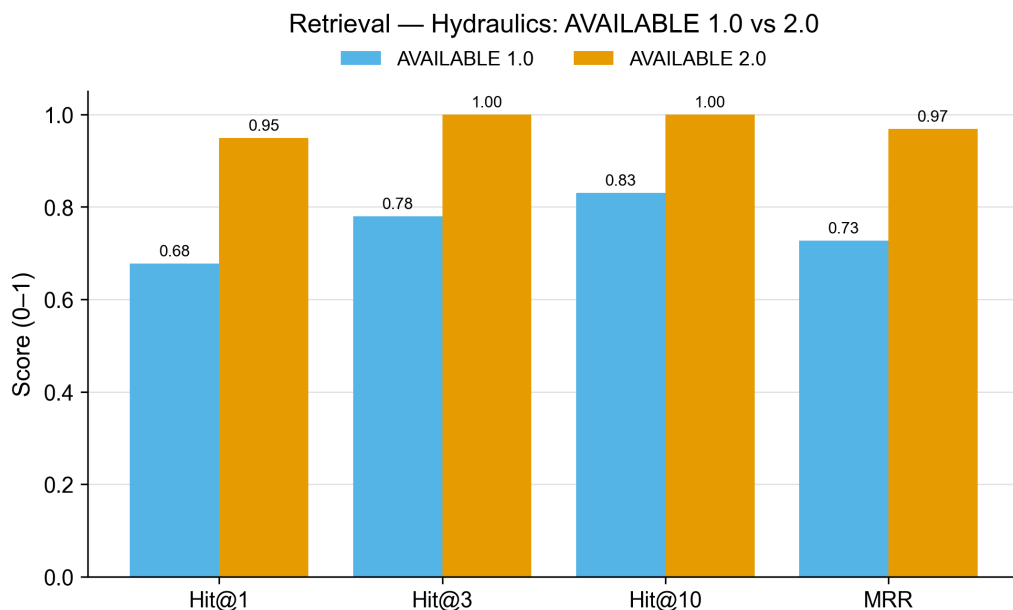


Figure 6.5: Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Hydraulics.

On the synthetic set the gain is smaller but consistent (Figure 6.6). Hit@1 rises from 0.56 to 0.68, Hit@10 from 0.83 to 0.92, and the mean reciprocal rank from 0.65 to 0.77. A likely reason for the smaller gain is that these questions were generated from the documents themselves and therefore share much of their wording, which favours a simple similarity search. The baseline is already strong here, so there is less room to improve.

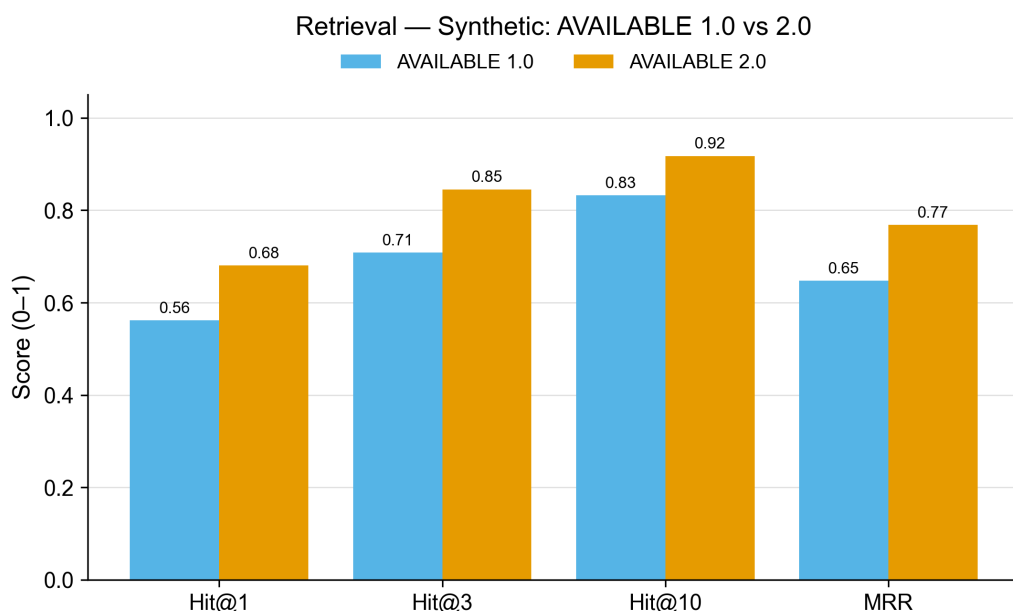


Figure 6.6: Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on the synthetic set.

Across the three datasets the picture is the same. AVAILABLE 2.0 is better on every dataset and on every measure. The largest gains are at the top of the ranking, which

matters most in practice, since passages at the top of the context are the ones that the generator uses best [21]. The wider reach also removes a hard limit. A document that never enters the window can never be used, and the second version reduces the share of questions without a correct document in the top ten from between 17 and 26 percent to between 0 and 8 percent.

This comparison measures the redesign as a whole. The new embedding model, the new vector store, the graph leg, the fusion and the reranker all changed together, so the figures show what the second version achieves but not how much each change contributed.

6.4 Generation Results

Generation is scored with the four RAGAS measures introduced in the methodology. Two of them judge the answer, which are faithfulness and answer relevancy, and two judge the retrieved context, which are context precision and context recall. The language model that writes the answer is the same in both versions, so any difference comes from the context that it receives.

On Grownfield, the context measures improve the most (Figure 6.7). Context precision rises from 0.36 to 0.77 and context recall from 0.52 to 0.80. This follows from the retrieval changes. The reranked pool puts relevant passages at the top, which raises precision, and the wider reach of the hybrid setting brings in more of the information that is needed, which raises recall. The answer measures improve by less, with faithfulness rising from 0.83 to 0.87 and answer relevancy from 0.86 to 0.91. The baseline answers were already reasonably faithful, and the better context mainly removes noise.

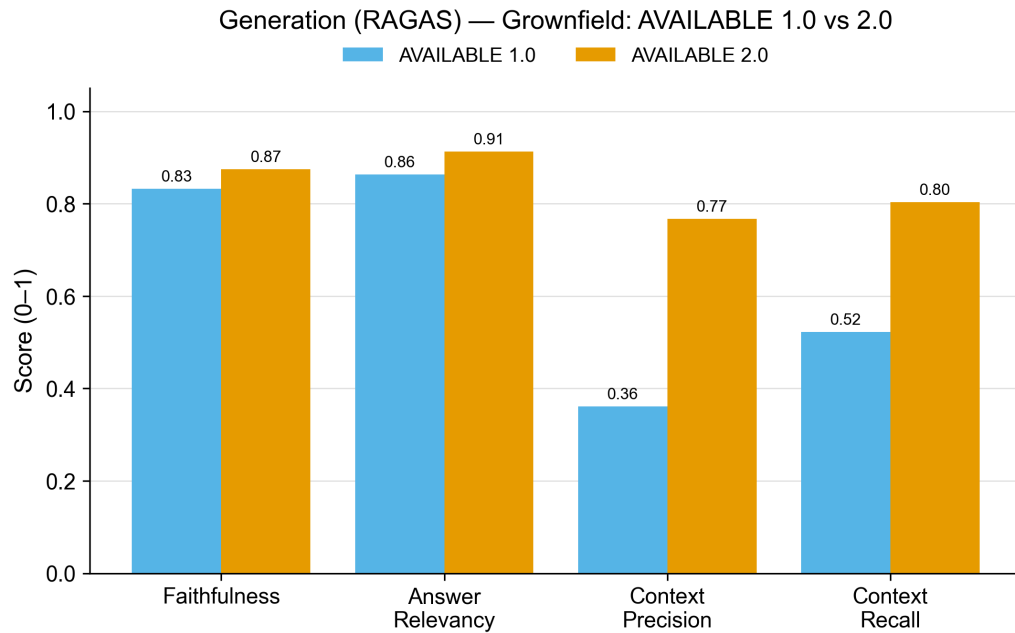


Figure 6.7: RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Grownfield.

Hydraulics shows the largest gains in the answer measures (Figure 6.8). Faithfulness rises from 0.57 to 0.69 and answer relevancy from 0.54 to 0.78, and context precision doubles from 0.32 to 0.68. Context recall rises from 0.41 to 0.59. These are also the lowest scores of the three datasets for both versions. This is notable because retrieval finds the golden document for every question here, with a Hit@3 of 1.00. The remaining limit on this dataset is therefore not finding the right document. It may lie in which parts of the document reach the model, or in how a judge model scores German text, but neither is tested here.

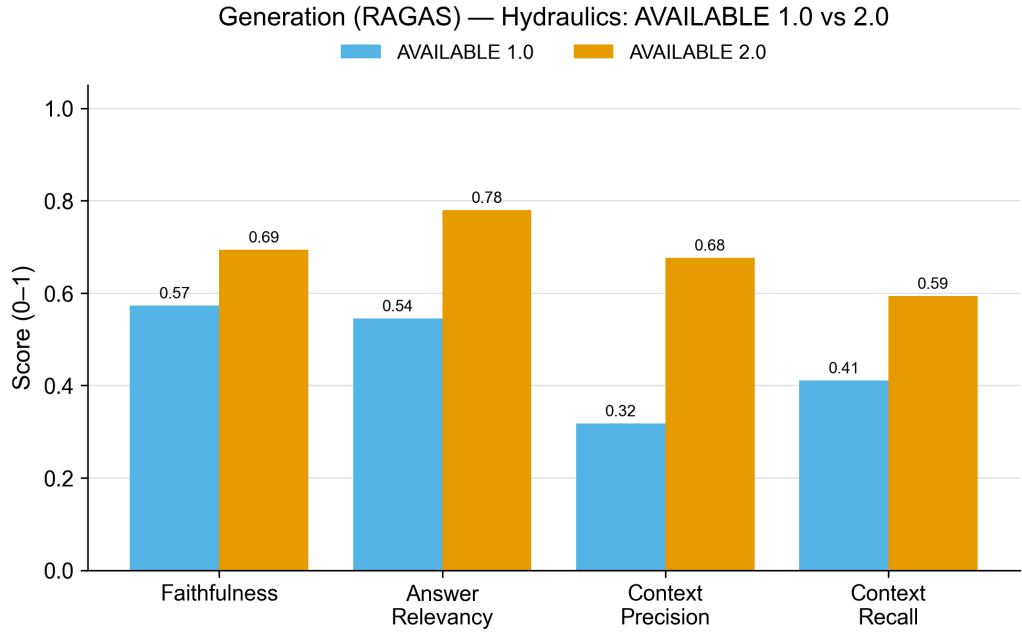


Figure 6.8: RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Hydraulics.

On the synthetic set both versions score high on the answer measures, and faithfulness hardly moves, from 0.87 to 0.88 (Figure 6.9). Answer relevancy rises from 0.92 to 0.95. The context measures still improve, with context precision rising from 0.66 to 0.84 and context recall from 0.86 to 0.94. The baseline already worked well on these questions, so there is little room left to improve the answers.

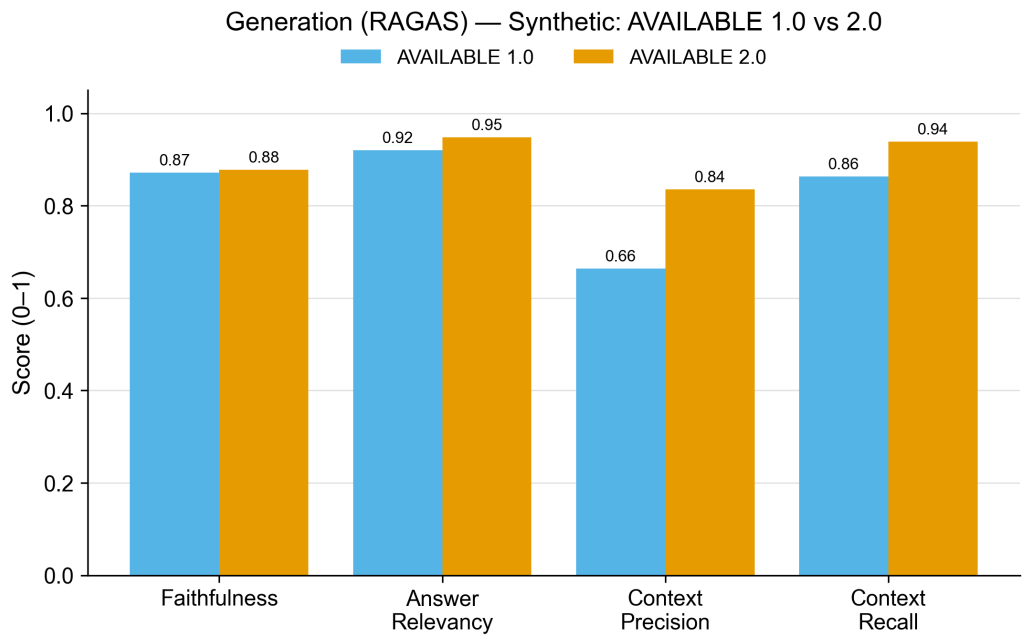


Figure 6.9: RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on the synthetic set.

Across the three datasets, the context measures improve the most and the answer measures improve by less. Better context therefore helps the answer most where the baseline struggled, which is on the two expert sets.

6.5 Trust Score Validation

The trust score gives each document a reliability value before any question is asked, as described in the trust scoring section of the methodology. The score has to prove two things. It must move in the right direction when a document loses quality, and it must help the system choose well when two documents cover the same topic but only one of them is reliable. Two experiments test these claims in turn.

Experiment 1 tests validity. Good documents are damaged in controlled ways, the score is computed again, and the question is whether it notices the damage and points to the part that was harmed. Experiment 2 tests usefulness. Reliable documents are paired with poisoned twins that share the topic but carry wrong facts. The score is switched on inside the retrieval and generation pipeline, and the question is whether it steers the system toward the correct source and the correct answer.

6.5.1 Experiment 1, Detecting a Loss of Quality

Four checks are made on the damaged copies described in the methodology. Only the prior is examined in this experiment. Each table reads one property that the score should have.

Table 6.1: Score of the targeted pillar before and after each attack.

Attack	Target pillar	Before	After	Change	Drop
Procedural	Process completeness	0.750	0.682	−0.068	−9.0%
Technical	Technical depth	0.825	0.708	−0.118	−14.2%
Filler	Focus	0.892	0.767	−0.125	−14.0%
Authority	Authority	0.932	0.300	−0.633	−67.8%
Freshness	Freshness	0.914	0.363	−0.551	−60.3%

Table 6.1 tests the most basic question. When an attack targets one pillar, does that pillar fall. It does in every case, so the score reacts to the kind of damage that it is meant to catch. The two metadata attacks fall the most, because they change a field outright and leave nothing behind to hold the score up. The content attacks fall less, since a language model that rewrites a document still leaves some real quality in the text. The score drops without going to the floor. This first check matters most, because a quality signal that did not move under real damage would be no signal at all.

Table 6.2: Overall prior before and after each attack.

Attack	Before	After	Change	Drop
Procedural	0.872	0.848	-0.023	-2.7%
Technical	0.872	0.837	-0.034	-3.9%
Filler	0.872	0.822	-0.049	-5.7%
Authority	0.872	0.714	-0.158	-18.1%
Freshness	0.872	0.789	-0.083	-9.5%

Table 6.2 moves from a single pillar to the full prior. The goal is to see whether one damaged pillar drags the whole score down or only part of it. Every prior falls by less than its pillar did, because the other pillars hold up the rest of the score. How far the prior falls follows the weight of the damaged pillar. For the authority attack the pillar fell by 0.633 and has a weight of 0.25, which gives $0.25 \times 0.633 = 0.158$, exactly the change in the table. The freshness attack gives $0.15 \times 0.551 = 0.083$ in the same way. Replacing a known author with an unknown one therefore pulls the prior down by a clear amount, which is the right reaction for Bosch training material, where documents from approved internal sources should count for more than unknown ones. The wider point is that a document with one weak spot loses some trust and not all of it, so the score does not treat a small fault as a total failure.

Table 6.3: Change of each content sub-score under the three content attacks. The largest fall in each row is in bold.

Attack	Technical depth	Process completeness	Focus
Procedural	-0.035	-0.068	-0.040
Technical	-0.118	-0.065	-0.043
Filler	-0.083	-0.113	-0.125

Table 6.3 asks whether the damage stays where it was aimed. For each content attack the biggest fall lands on the sub-score that it was meant to hurt. This shows that the sub-scores measure separate things and not one blurred sense of quality that slides together no matter what is broken. The other sub-scores also fall to some degree, which is expected, since a damaged text is rarely damaged in one way only. The filler attack is the weakest case, where the fall in process completeness, at 0.113, is almost as large as the fall in focus, at 0.125. Even so, the score can point to what is mostly wrong with a document and does not only report that it is worse.

Table 6.4: Wilcoxon signed rank tests of the targeted content sub-scores. The column p_{BH} gives the p-value after the Benjamini Hochberg correction for the three tests.

Attack	Dropped	Median change	95% CI	d	p_{BH}
Procedural	20 of 40	-0.050	[-0.100, -0.037]	-0.68	0.0001
Technical	29 of 40	-0.100	[-0.155, -0.083]	-1.00	< 0.0001
Filler	25 of 40	-0.100	[-0.180, -0.075]	-0.72	0.0001

Table 6.4 confirms that the content drops are real and not chance. Every content attack lowers its targeted sub-score with a clear effect, and the effect sizes are medium to large, with Cohen’s d between 0.68 and 1.00. The technical attack is the strongest of the three. The procedural attack is the weakest, since its sub-score fell in only half of the documents. The confidence intervals stay clear of zero throughout, and the result holds after correcting for the three tests that were run together. The metadata attacks need no test, since they change a field outright and their effect is certain by construction. This gives the earlier tables statistical backing, so the drops can be read as the score working and not as noise in the scorer.

Across all four checks the score behaves as a valid signal should. It reacts to deliberate quality loss, spreads that loss sensibly across the prior, keeps it mostly local to the right sub-score, and does so with statistical support. Validity is established. That is the precondition for the next question, whether a valid score is also useful once it is switched on inside the live system.

6.5.2 Experiment 2, Helping the RAG

The setup is the corpus of good documents and poisoned twins described in the methodology. The score is read at two layers, retrieval and generation, across three arms. In the first arm trust is off, in the second only the prior is used, and in the third the full score of prior and support is used.

6.5.2.1 Retrieval

Table 6.5: Ranking of the good document against its poisoned twin. The best value in each column is in bold.

Arm	N	Good above twin	Twin above good	Neither	Good first
Trust off	190	73	82	35	38.4%
Prior	190	135	20	35	71.1%
Full score	190	124	31	35	65.3%

Table 6.5 asks the first question. When a good document and its poisoned twin both reach the pool, does the good one rank higher. With trust off the answer is often no. The good source leads for well under half of the questions, and the twin wins more often than the good document does. This is the exact failure that the trust score exists to fix, because the twin shares the topic and the wording, so plain relevance cannot tell the two apart. Turning on the prior changes the picture sharply. The share of questions where the good document comes first almost doubles, and the wins of the twin drop to a fraction of what they were. The full score helps too, but it lands below the prior. One block of 35 questions stays the same across all three arms, the ones where neither document is retrieved at all. Trust can reorder the pool, but it cannot bring back a document that retrieval never found.

Table 6.6: McNemar tests of the retrieval result. The column b counts questions that the first arm gets right and the second gets wrong, and c counts the opposite.

Comparison	b	c	χ^2	p (exact)	Better
Off versus prior	0	62	60.016	< 0.0001	Prior
Prior versus full	11	0	9.091	0.0010	Prior
Off versus full	4	55	42.373	< 0.0001	Full

Table 6.6 checks these shifts with a paired test. Both trust arms beat the off arm by a wide and clear margin. The split between off and prior runs one way only. Many questions flip to the good source once the prior is on, and none flip back. The comparison of prior and full favours the prior, since adding the support score pushes some questions back to the twin and rescues none. At the retrieval layer the prior therefore carries the whole benefit, and the support score takes a little of it away.

6.5.2.2 Generation

Table 6.7: Share of correct answers by grader and arm. The best value per row is in bold, except for RAGAS, which was found to be unreliable.

Grader	Trust off	Prior	Full score
Exact match	63.2%	70.0%	64.2%
LLM judge	41.1%	70.0%	63.7%
RAGAS	21.1%	18.9%	19.5%

Table 6.7 moves to the written answer. The question now is whether the answer follows the correct value or the wrong value of the twin. Two of the three graders, exact match and the language model judge, put the prior arm highest and agree closely on the two trust arms. They differ on the arm with trust off, for a reason that Table 6.9 makes

clear. RAGAS sits far lower and hardly moves across the arms, which is a warning that this grader is not reading the task.

Table 6.8: Agreement of the two model based graders with the known correct value. The count is 190 questions in three arms.

Grader	N	Agreement with ground truth
LLM judge	570	86.1%
RAGAS	570	46.0%

Table 6.8 follows up on that warning by checking each grader against the known correct value. The language model judge agrees with the ground truth on most answers. RAGAS agrees on less than half, close to a coin flip. RAGAS answer correctness is built for longer generated text, and it misreads the short factual answers used here. For that reason exact match is kept as the main grader and RAGAS is set aside. This step matters, because trusting RAGAS would have buried the real effect under a grader that cannot see it.

Table 6.9: How the answers of each arm fall into four outcomes, out of 190 questions. The best value in the first three columns is in bold.

Arm	Correct	Poisoned	Both	Missing
Trust off	78	17	42	53
Prior	132	7	1	50
Full score	118	13	4	55

Table 6.9 splits each arm into the four ways an answer can land. It can name the correct value only, the wrong value only, both values, or neither. The exact match grader counts an answer as correct when it contains the correct value, so the hedged answers in the third column also count as correct under it. This is why the arm with trust off scores 63.2% in the exact match row, although only 78 of its 190 answers name the correct value alone. The prior gives the largest gain. Answers that give only the correct value rise by a big step, and answers that follow the wrong value fall to a handful. The clearest shift is in the third column. With trust off the model often hedges and names the right value and the wrong value together instead of choosing. Once the prior tags the trusted chunk in the prompt, that hedging almost vanishes and the model commits to the correct source. This is the practical payoff. Trust does not only reorder documents, it gives the generator a reason to pick one source when two disagree. The full score again improves on the baseline without trust but trails the prior.

Table 6.10: McNemar tests of the answer correctness under exact match.

Comparison	<i>b</i>	<i>c</i>	χ^2	<i>p</i> (exact)	Better
Off versus prior	4	17	6.857	0.0072	Prior
Prior versus full	13	2	6.667	0.0074	Prior
Off versus full	13	15	0.036	0.8506	No difference

Table 6.10 tests the answer outcomes with the same paired test. The prior significantly improves correctness over the baseline without trust. The prior also beats the full score, since a group of questions are correct under the prior but wrong once the support score is added. The comparison of off and full is not significant, and that is the sharpest result in the table. On its own, the full score does not reliably beat using no trust at all.

Both layers read the same way. The prior, built from content and metadata, does the useful work. It ranks the trusted source above its twin and steers the answer to the right value. The support score adds nothing beyond the prior and takes a little away. The likely reason lies in the test itself, as discussed in the next subsection.

6.5.3 Why the Full Score Trails the Prior

A twin is almost a copy of its good document. The attack flips about ten facts and weakens the metadata, and the rest stays the same, so the two documents look nearly identical. The prior reads the document itself, its weak metadata and its content, which is where the good document and the twin differ, so it separates them cleanly. The support score measures corroboration, meaning how well a document connects to other trusted documents, and it checks agreement and not truth, as discussed in the limits of the method in the methodology. Because the twin is a near copy, it probably sits in the same part of the graph as the original and looks just as well supported, so the support score cannot tell them apart. Blending the two then leaves the full score weaker than the prior alone, which is why some orderings and answers fall back to the wrong source once the support score is added. This explanation fits the data, but it was not tested separately here. It also fits the choice made in the methodology to give the support score a weight of only 0.3, so that it can nudge a score but not override the prior.

One point keeps this in perspective. This is the hardest case for the support score by design, since the twins are near duplicates. For a truly isolated junk document, with no shared terms and no graph links, the support score would be expected to give a low value and to help. This experiment does not test that case. Against copy and paste poisoning it cannot help, because agreement alone cannot punish a wrong fact that reuses real terms.

6.6 Observability Dashboard

7 Discussion

8 Conclusion

9 Future Work

Bibliography

- [1] World Economic Forum, “The Future of Jobs Report 2025,” Geneva, Switzerland, technical report, 2025. [Online]. Available: <https://www.weforum.org/reports/the-future-of-jobs-report-2025/>
- [2] W. Zornek, “Welcome to the VUCA (BANI) World,” in *Strategic Management Reloaded*, Munich: Haufe, 2024. doi: 10.34157/978-3-648-17700-6_2.
- [3] L. Pundt, “Demographic Change and Demographic Management,” in *Organizational Psychology for Human Resource Management*, B. Werkmann-Karcher, A. Müller, and T. Zbinden, Eds., Berlin, Heidelberg: Springer, 2023. doi: 10.1007/978-3-662-65308-1_4.
- [4] M. Dommermuth, “Modular Learning Factories for Industry 4.0. Acquisition of Target-Oriented Action Competence to Accelerate Industrial Implementation,” *Industrie 4.0 Science*, pp. 25–30, 2024, doi: 10.30844/I4SE.24.4.24.
- [5] M. Dommermuth, “Ready for the Digital Transformation? Significant Skill Gaps and Learning Needs in the Industry Require New Approaches,” *ZWF Zeitschrift für wirtschaftlichen Fabrikbetrieb*, vol. 119, pp. 460–464, 2024, doi: 10.1515/zwf-2024-1083.
- [6] W. Fan *et al.*, “A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models,” in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6491–6501.
- [7] B. Tural, Z. Örpek, and Z. Destan, “Retrieval-Augmented Generation (RAG) and LLM Integration,” in *International Symposium on Innovative Approaches in Smart Technologies (ISAS)*, 2024.
- [8] M. Dommermuth, L. Riesch, and M. A. Guedria, “Human-Centered AI Chatbot for the Personalized Delivery of Trustworthy Learning Content,” in *GfA Frühjahrskongress 2025, Aachen. Arbeit 5.0: Menschzentrierte Innovationen für die Zukunft der Arbeit*, Gesellschaft für Arbeitswissenschaft (GfA), 2025, pp. 963–968.
- [9] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024.

- [10] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474.
- [11] W. Zhu, X. Zhang, Q. Zhai, and C. Liu, “A Hybrid Text Generation-Based Query Expansion Method for Open-Domain Question Answering,” *Future Internet*, vol. 15, no. 5, p. 180, 2023, doi: 10.3390/fi15050180.
- [12] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [13] D. Edge *et al.*, “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [14] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval,” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [15] H. Sun, B. Dhingra, M. Zaheer, K. Mazaitis, R. Salakhutdinov, and W. W. Cohen, “Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 4231–4242.
- [16] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [17] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” technical report 1999–66, 1999.
- [18] T. H. Haveliwala, “Topic-Sensitive PageRank,” in *Proceedings of the 11th International Conference on World Wide Web (WWW)*, 2002, pp. 517–526.
- [19] B. J. Gutiérrez, Y. Shu, M. Yasunaga, and Y. Su, “From RAG to Memory: Non-Parametric Continual Learning for Large Language Models,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [20] B. Sarmah, B. Hall, R. Rao, S. Patel, S. Pasquali, and D. Mehta, “HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,” in *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF)*, 2024, pp. 608–616.

- [21] N. F. Liu *et al.*, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [22] Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [23] Z. Gyöngyi, H. Garcia-Molina, and J. Pedersen, “Combating Web Spam with TrustRank,” in *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, 2004, pp. 576–587.
- [24] H. Qian, B. Li, and Q. Wang, “ClaimTrust: Propagation Trust Scoring for RAG Systems,” *arXiv preprint arXiv:2503.10702*, 2025.
- [25] A. Jia, A. Ramesh, Z. Shamsi, D. Zhang, and A. Liu, “RAGRank: Using PageRank to Counter Poisoning in CTI LLM Pipelines,” in *Annual Computer Security Applications Conference (ACSAC), Poster*, 2025.
- [26] T. Yu *et al.*, “Building Trustworthy Knowledge Retrieval for Nuclear Chemical Engineering: A Multi-Dimensional Trust Evaluation and Propagation Framework,” technical report, 2025.
- [27] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, “Evaluation of Retrieval-Augmented Generation: A Survey,” in *Big Data (BigData 2024)*, vol. 2301, in *Communications in Computer and Information Science*, vol. 2301., Springer, 2025.
- [28] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [29] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [30] D. Moshkovich, H. Mulian, S. Zeltyn, N. Eder, I. Skarbovsky, and R. Abitbol, “Beyond Black-Box Benchmarking: Observability, Analytics, and Optimization of Agentic Systems,” *arXiv preprint arXiv:2503.06745*, 2025.
- [31] D. Kreuzberger, N. Köhl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
- [32] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

- [33] V. Karpukhin *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.
- [34] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [35] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [36] A. Hogan *et al.*, “Knowledge Graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [37] K. Spärck Jones, “A Statistical Interpretation of Term Specificity and Its Application in Retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [38] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” *arXiv preprint arXiv:1901.04085*, 2019.
- [39] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, in Proceedings of Machine Learning Research, vol. 202. 2023, pp. 28492–28518.
- [40] Z. Zhong, H. Liu, X. Cui, X. Zhang, and Z. Qin, “Mix-of-Granularity: Optimize the Chunking Granularity for Retrieval-Augmented Generation,” *arXiv preprint arXiv:2406.00456*, 2024.
- [41] A. Panickssery, S. R. Bowman, and S. Feng, “LLM Evaluators Recognize and Favor Their Own Generations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

List of Figures

Figure 5.1	AVAILABLE 1.0 architecture, with document preparation above and query handling below.	23
Figure 5.2	AVAILABLE 2.0 architecture with the data ingestion, retrieval and generation, and observability and evaluation pipelines.	24
Figure 5.3	The data preparation pipeline, organised as a medallion architecture. Raw training files are collected from several SharePoint libraries into a synchronised local mirror (Bronze), extracted and standardised into uniform structured records using GPT-4o and Whisper (Silver), and assembled into per-training collections of chunks (Gold). The Gold chunks are the input to the retrieval stage, where they are embedded into a vector index and arranged into a knowledge graph.	27
Figure 5.4	Schema of the knowledge graph with its structural layer and its semantic layer. The table lists the edge weights used during traversal.	36
Figure 5.5	Shared seed generation for both graph engines, with the entity hop on the left and Personalized PageRank on the right. Circles are entities, squares are chunks, and the numbers are the weights that reach each node.	38
Figure 5.6	The two eye retrieval flow from the two legs to the final top-k chunks. ...	43
Figure 5.7	Computation of the trust score. The prior is built from four pillars, the support score spreads trust across a document graph, and the final score is stored once at ingestion and read back at query time.	45
Figure 6.1	Graph retrieval quality by traversal engine, with identical entity seeds for all three engines.	58
Figure 6.2	Retrieval quality of the vector, graph and hybrid legs, measured by Hit@10 and mean reciprocal rank.	59
Figure 6.3	RAGAS generation quality of the hybrid leg on Grownfield.	60
Figure 6.4	Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Grownfield.	61
Figure 6.5	Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on Hydraulics.	62
Figure 6.6	Retrieval quality of AVAILABLE 1.0 and AVAILABLE 2.0 on the synthetic set.	62

Figure 6.7 RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on
Grownfield. 64

Figure 6.8 RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on
Hydraulics. 65

Figure 6.9 RAGAS generation quality of AVAILABLE 1.0 and AVAILABLE 2.0 on
the synthetic set. 65

List of Tables

Table 5.1	Comparison of AVAILABLE 1.0 and AVAILABLE 2.0.	24
Table 5.2	Distribution of the corpus across topics.	26
Table 5.3	Distribution of the corpus across file formats.	26
Table 5.4	Baseline and enhanced retrieval representation.	33
Table 5.5	Overview of the two evaluation datasets.	49
Table 5.6	The five kinds of damage used in the first experiment.	52
Table 5.7	Writing guidelines derived from the three parts of the trust score and their share of the final score.	56
Table 6.1	Score of the targeted pillar before and after each attack.	66
Table 6.2	Overall prior before and after each attack.	67
Table 6.3	Change of each content sub-score under the three content attacks. The largest fall in each row is in bold.	67
Table 6.4	Wilcoxon signed rank tests of the targeted content sub-scores. The column p_{BH} gives the p-value after the Benjamini Hochberg correction for the three tests.	68
Table 6.5	Ranking of the good document against its poisoned twin. The best value in each column is in bold.	68
Table 6.6	McNemar tests of the retrieval result. The column b counts questions that the first arm gets right and the second gets wrong, and c counts the opposite.	69
Table 6.7	Share of correct answers by grader and arm. The best value per row is in bold, except for RAGAS, which was found to be unreliable.	69
Table 6.8	Agreement of the two model based graders with the known correct value. The count is 190 questions in three arms.	70
Table 6.9	How the answers of each arm fall into four outcomes, out of 190 questions. The best value in the first three columns is in bold.	70
Table 6.10	McNemar tests of the answer correctness under exact match.	71